MACHINE LEARNING PORTFOLIO · MSC DATA SCIENCE · ARDEN UNIVERSITY BERLIN

Product Segmentation and Performance Clustering for an E-Commerce Fragrance Marketplace

An end-to-end unsupervised machine learning investigation comparing K-Means, Ward hierarchical clustering and DBSCAN across 1,000 marketplace listings, translated into evidence-based actions for catalogue strategy.

STUDENT ID	**25199053**
MODULE	COM7022 Machine Learning · Assignment [5444]
ASSESSMENT	Portfolio · Task 1 Practical Project · Task 2 Critical Report
WORD COUNT	2,870 words of body text (Task 1: 2,036 · Task 2: 834); captions, tables, figures, code screenshots, references and appendix excluded
DATASET	Sales-[5444].csv · 1,000 listings × 10 attributes
REPOSITORY	https://github.com/kvmmn/_COM7022_MachineLearning
REVIEW PORTAL	https://kvmmn.github.io/_COM7022_MachineLearning/

CORE REVENUE ENGINE	INVENTORY RISK	MODEL STABILITY	PCA VARIANCE
92.3%	**31.3%**	**0.932**	**78.7%**
of units sold come from 24.3% of listings (Cluster 2)	of listings sit in the low-stock tail (Cluster 3)	K-Means bootstrap ARI over 30 resamples	retained by the first two components

Every figure, table and code screenshot carries a caption that names and links its exact file in the GitHub repository.

Contents

1. Introduction 3

1.1 Business Context 3

1.2 The Dataset 3

1.3 Problem Statement 3

1.4 Project Objectives 3

1.5 Methodological Approach 3

TASK 1 · PRACTICAL MACHINE LEARNING INVESTIGATION

2. Data Preparation 5

2.1 Data Quality Audit 5

2.2 Cleaning Strategy 6

2.3 Transformation, Scaling and Categorical Variables 9

2.4 Feature Selection 10

3. Model Selection 11

3.1 Clustering in Brief 11

3.2 Candidate Techniques 12

3.3 Dimensionality Reduction with PCA 12

4. Model Implementation and Evaluation 13

4.1 K-Means 13

4.2 Hierarchical Clustering 14

4.3 DBSCAN 15

4.4 Evaluation Framework 17

4.5 Comparative Results 17

5. Cluster Characteristics 18

TASK 2 · CRITICAL REPORT FOR EXECUTIVE STAKEHOLDERS

6. Executive Summary 20

7. Critical Evaluation 20

7.1 Suitability of the Methods and Design Decisions 20

7.2 Comparative Performance: Strengths and Limitations 20

7.3 Why K-Means Is the Right Choice for E-Commerce 20

8. Recommendations for Business Managers 21

9. Limitations 22

10. Conclusion and Future Work 22

References 23

Appendix A: Reproducible Code 24

1. Introduction

1.1 Business Context

This portfolio is written from the position of a newly appointed Machine Learning Analyst at a rapidly growing e-commerce marketplace. The platform operates a multi-seller model: independent merchants list competing offers for the same products, so the catalogue expands far faster than any merchandising team can curate by hand. Commercial decisions that once relied on manual review (which listings to promote, which to replenish, which to retire) now require a scalable, data-driven view of the catalogue.

1.2 The Dataset

The analysis uses the company extract `Sales-[5444].csv`: **1,000 listings described by 10 attributes** covering product identity (`Item_Name`, `details`, `Catatory`), pricing (`price`, `priceWithCurrency`), availability and traction (`available`, `text`, `sold`), maintenance history (`lastUpdated`) and dispatch origin (`itemLocation`). The catalogue is a men's fragrance category, dominated by US-based sellers (909 listings) with a smaller cross-border segment led by Hong Kong (52). Critically, each row is a **seller listing rather than a unique product**: the same fragrance can appear several times at different prices and stock positions, so the resulting segments describe commercial offers, not manufacturers.

1.3 Problem Statement

Three obstacles prevent the company from managing this catalogue strategically. First, **scale**: manual review cannot keep pace with listing growth. Second, **data quality**: prices and sales are partly stored as free text, category labels are inconsistent and the platform censors stock above ten units. Third, **the absence of labels**: no field states whether a listing is a best-seller, a premium niche item or an inventory risk, which rules out supervised learning. The business question is therefore one of *discovery*: what natural commercial groups exist in the catalogue, and how should each be managed?

1.4 Project Objectives

- **O1 — Data readiness:** audit, clean and transform the dataset with every intervention evidenced and reproducible.
- **O2 — Segmentation:** apply K-Means, Hierarchical (agglomerative) clustering and DBSCAN to group listings by product characteristics and sales performance.
- **O3 — Rigorous evaluation:** compare models with complementary internal, stability and sensitivity measures rather than a single score.
- **O4 — Model justification:** select the model that is technically strongest *and* operationally suited to e-commerce.
- **O5 — Business translation:** convert segments into evidence-based actions for non-technical decision-makers.

1.5 Methodological Approach

The investigation follows a structured machine learning lifecycle (Figure 1). Each stage is implemented in a single Google Colab notebook with fixed random seeds, and every analytical decision is logged so that results can be regenerated end to end. The notebook was re-executed from a clean state before this report was written and reproduced every reported figure.

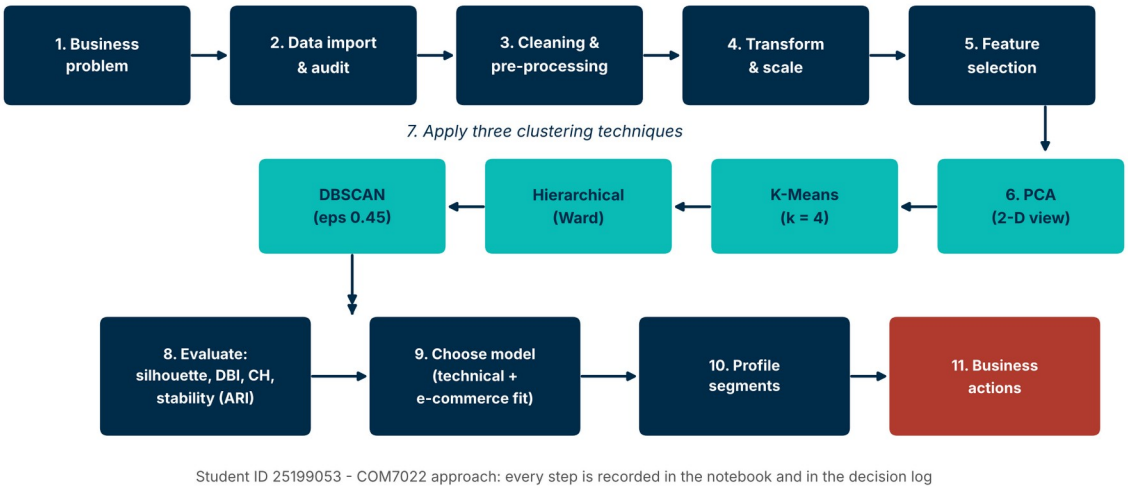


Figure 1. End-to-end methodological workflow, from business problem to recommended actions.
Source: COM7022 - Assignment/output/figures/fig15_workflow.png

TASK 1

Practical Machine Learning Investigation

2,036 words of body text

2. Data Preparation

2.1 Data Quality Audit

A systematic audit preceded any modelling. The file contains **no full-row duplicates**, although 34 listings repeat an identical description, consistent with multi-seller offers rather than data-entry error. Missing values affect eight columns (Table 1, Figure 2); the largest gaps are in `available` (111 rows, 11.1%) and `lastUpdated` (53 rows, 5.3%).

Descriptive profiling exposed four structural problems that would distort any distance-based algorithm:

- **Extreme skew.** Units sold range from 1 to 54,052 against a median of 49.5 (skewness 9.70); price is also right-skewed (2.04). Euclidean distances on raw values would be dominated by a handful of viral listings (Figure 3).
- **Censored stock.** The platform displays "More than 10 available" for 299 listings and stores the value as 10, so recorded stock is a ceiling, not a measurement.
- **Label fragmentation.** `Catatory` holds 63 distinct strings that are largely spelling and capitalisation variants of a few fragrance types.
- **Mixed currency.** Three prices are quoted in Canadian dollars.

Table 1. Missing values per attribute in the raw extract (n = 1,000).

Attribute	Missing	Missing %
available	111	11.1
lastUpdated	53	5.3
sold	8	0.8
price	5	0.5
Catatory	3	0.3
text	3	0.3
Item_Name	1	0.1
priceWithCurrency	1	0.1

Source: COM7022 - Assignment/output/tables/table01_missing_values.csv

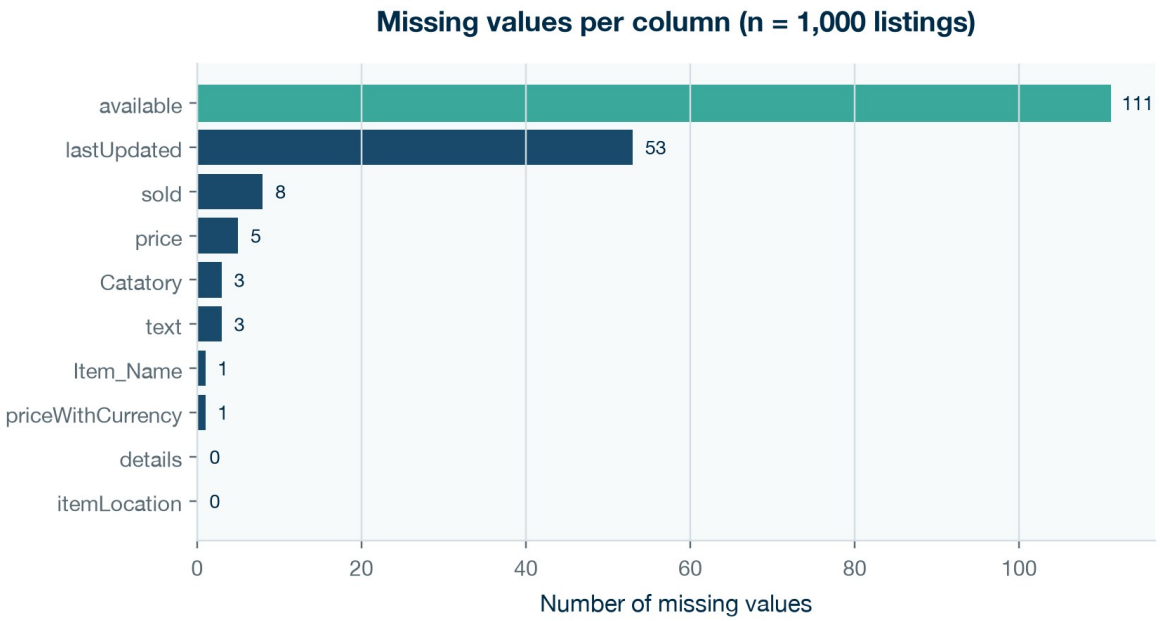


Figure 2. Missing values by attribute in the raw extract.
Source: COM7022 - Assignment/output/figures/fig01_missing_values.png

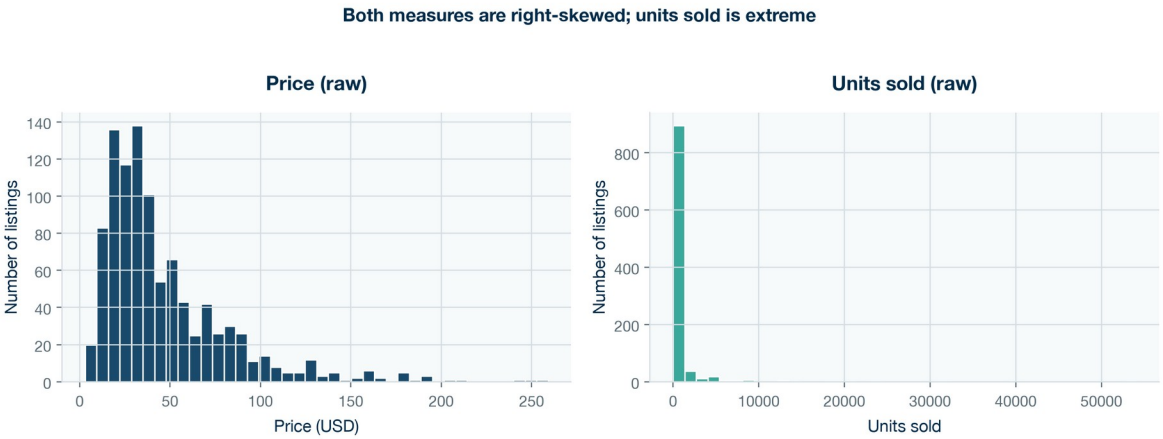


Figure 3. Raw distributions of price and units sold, showing the extreme right skew.
Source: COM7022 - Assignment/output/figures/fig02_raw_distributions.png

2.2 Cleaning Strategy

Preparation followed a single principle: **recover before you impute**. Where a value was absent, it was first reconstructed from another field of the same listing; statistical imputation was used only where the cause of the gap was understood, and it was always flagged. Every step was recorded in an execution log (Table 2) and the overall flow is summarised in Figure 4.

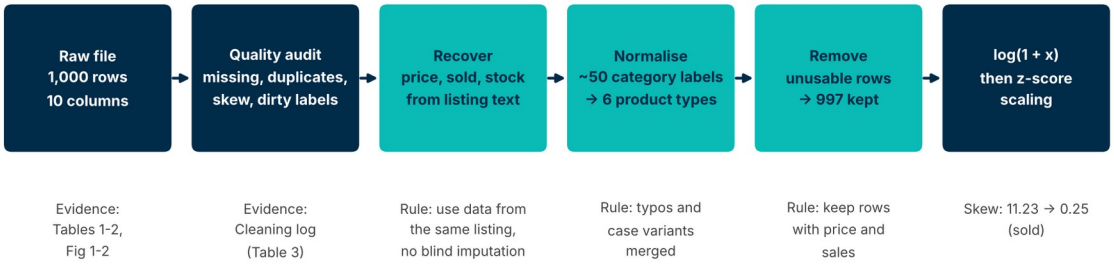


Figure 4. Data preparation pipeline from the raw extract to model-ready features.
Source: COM7022 - Assignment/output/figures/fig16_data_prep_flow.png

- **Price and sales recovery.** All five missing prices were restored from `priceWithCurrency`, which matched the numeric field exactly across all 994 rows holding both. Two missing sales figures were parsed from the `text` field, and three listings showing stock but no sales were set to zero. Canadian prices were converted at a documented fixed rate.
- **Stock reconstruction.** Availability was rebuilt from listing wording: "More than 10" and any explicit value above ten were coded as 11 (*well stocked*), "Last One" as 1, and the 62 "Limited quantity" or lot listings received the median with an indicator column, so their influence could be audited.
- **Category normalisation.** The 63 raw labels were consolidated into six coherent product types (Eau de Toilette, Eau de Parfum, Cologne, Parfum/Extrait, Generic fragrance, Other).
- **Record retention.** Only three listings with neither sales nor stock information were removed, leaving **997 verified listings**. No commercially extreme listing was discarded, because best-sellers are precisely the observations the business must understand.

Table 2. Execution log of every cleaning and transformation step.

Pre-processing step	Rows affected	Rationale
Recover price from priceWithCurrency	5	Exact same value, only stored in text form
Convert CAD prices to USD	3	Fixed rate 0.73, documented approximation
Recover sold from text	2	Exact value from the listing wording
Set sold = 0 where text shows stock but no sales	3	Listing wording omits 'sold' when nothing has sold
Drop rows with no sales or stock information	3	Cannot be placed on the sales-performance axis
Stock: 'More than 10' coded as 11	299	Censored value, not exact
Stock: explicit values above 10 capped at 11	115	Same scale as the censored rows
Stock: 'Last One' coded as 1	47	Wording means one unit left
Stock: median imputation	62	Limited quantity / lot wording; flagged in stock_imputed
Collapse category labels into 6 product types	994	Case, spelling and wording variants
Missing brand set to 'Unbranded'	1	Single row
Log-transform price and sold	997	Outliers kept; log scale limits their pull on distances

Source: COM7022 - Assignment/output/tables/table03_cleaning_log.csv

COM7022_Clustering_25199053.ipynb | Missing-value audit

```python
# Missing values per column, as counts and percentages
missing = pd.DataFrame({
    "missing_count": raw.isna().sum(),
    "missing_pct": (raw.isna().mean() * 100).round(1),
})
missing = missing.sort_values("missing_count", ascending=False)
print("Full-row duplicates:", raw.duplicated().sum())
print("Duplicate listing descriptions:", raw["details"].duplicated().sum())
save_table(missing, "table01_missing_values.csv")
missing
```

Figure 5. Code screenshot: Missing-value and duplicate audit.
Source: COM7022 - Assignment/output/screenshots/code02_missing_values.png

COM7022_Clustering_25199053.ipynb | Cleaning: recover price

```python
df = raw.copy()
cleaning_log = []


def log_step(step, rows_affected, note):
    cleaning_log.append({"step": step, "rows_affected": int(rows_affected), "note": note})


# 3.1 Price: recover missing prices from priceWithCurrency (same listing, same number)
price_from_text = df["priceWithCurrency"].str.extract(r"\$([\d,]+\.?\d*)")[0]
price_from_text = price_from_text.str.replace(",", "").astype(float)
recovered = df["price"].isna() & price_from_text.notna()
df.loc[recovered, "price"] = price_from_text[recovered]
log_step("Recover price from priceWithCurrency", recovered.sum(), "Exact same value, only stored in text form")

# Check: where both exist, do they agree?
both = df["price"].notna() & price_from_text.notna()
print("Price disagreements where both exist:", (abs(df.loc[both, "price"] - price_from_text[both]) > 0.01).sum())

# 3.2 Currency: convert the three Canadian-dollar prices to USD
# Approximate May 2024 rate (1 CAD ~ 0.73 USD). Only 3 rows, so the exact rate barely matters.
CAD_TO_USD = 0.73
is_cad = df["priceWithCurrency"].fillna("").str.startswith("C")
df.loc[is_cad, "price"] = (df.loc[is_cad, "price"] * CAD_TO_USD).round(2)
log_step("Convert CAD prices to USD", is_cad.sum(), "Fixed rate 0.73, documented approximation")
```

Figure 6. Code screenshot: Recovery of price from priceWithCurrency and currency conversion.
Source: COM7022 - Assignment/output/screenshots/code03_price_recovery.png

COM7022_Clustering_25199053.ipynb | Cleaning: recover units sold

```python
# 3.3 Units sold: recover from the 'text' column ("... / 116 sold")
sold_from_text = df["text"].str.extract(r"([\d,]+) sold")[0].str.replace(",", "").astype(float)
recovered = df["sold"].isna() & sold_from_text.notna()
df.loc[recovered, "sold"] = sold_from_text[recovered]
log_step("Recover sold from text", recovered.sum(), "Exact value from the listing wording")

# Check the two sources agree everywhere else
both = df["sold"].notna() & sold_from_text.notna()
print("Sold disagreements where both exist:", (df.loc[both, "sold"] != sold_from_text[both]).sum())

# Listings whose text shows stock but no 'sold' part have not sold yet, so sold = 0
no_sales_yet = df["sold"].isna() & df["text"].notna() & ~df["text"].str.contains("sold", na=False)
df.loc[no_sales_yet, "sold"] = 0
log_step("Set sold = 0 where text shows stock but no sales", no_sales_yet.sum(), "Listing wording omits 'sold' when nothing has sold")

# Rows with no text and no sold value carry no sales or stock information at all
no_info = df["sold"].isna() & df["text"].isna()
print(df.loc[no_info, ["Item_Name", "price", "available", "text", "sold"]])
df = df[~no_info].copy()
log_step("Drop rows with no sales or stock information", no_info.sum(), "Cannot be placed on the sales-performance axis")
```

Figure 7. Code screenshot: Recovery of units sold from the listing text.
Source: COM7022 - Assignment/output/screenshots/code04_sold_recovery.png

COM7022_Clustering_25199053.ipynb | Cleaning: stock level and censoring

```python
# 3.4 Stock level: rebuild it from the listing wording so every row has a value.
# The marketplace hides exact numbers above 10 ("More than 10 available"), so I cap all
# stock values at 11, meaning "more than 10". This keeps the measure honest and comparable.
text = df["text"].fillna("")
stock = text.str.extract(r"^(\d+) available")[0].astype(float)
stock[text.str.startswith("More than")] = 11
stock[text.str.startswith("Last One")] = 1
capped_above_10 = (stock > 11).sum()
stock = stock.clip(upper=11)

# "Limited quantity available" and lot listings give no number: impute the median, but keep a flag
stock_imputed = stock.isna()
df["stock_imputed"] = stock_imputed.astype(int)
stock = stock.fillna(stock.median())
df["stock_level"] = stock

log_step("Stock: 'More than 10' coded as 11", text.str.startswith("More than").sum(), "Censored value, not exact")
log_step("Stock: explicit values above 10 capped at 11", capped_above_10, "Same scale as the censored rows")
log_step("Stock: 'Last One' coded as 1", text.str.startswith("Last One").sum(), "Wording means one unit left")
log_step("Stock: median imputation", stock_imputed.sum(), "Limited quantity / lot wording; flagged in stock_imputed")
print(df["stock_level"].value_counts().sort_index())
```

Figure 8. Code screenshot: Stock reconstruction and treatment of censoring.
Source: COM7022 - Assignment/output/screenshots/code05_stock_level.png

COM7022_Clustering_25199053.ipynb | Cleaning: normalise product type

```python
# 3.5 Product type: collapse the messy category labels into six groups


def group_category(label):
    if pd.isna(label):
        return "Other"
    s = str(label).strip().lower()
    if s in ["", "/", "y", "does not apply", "unknown", "various", "assorted"]:
        return "Other"
    if "," in s or "/" in s:
        return "Other"                          # listings that name several types at once
    if "toil" in s or s == "edt":
        return "Eau de Toilette"                # includes the misspelling 'toillette'
    if "eau de parfum" in s or "eau de perfume" in s:
        return "Eau de Parfum"
    if "parfum" in s or "extrait" in s or "elixir" in s:
        return "Parfum / Extrait"
    if "cologne" in s or s == "edc":
        return "Cologne"
    if "perfume" in s or "fragrance" in s:
        return "Generic fragrance"
    return "Other"                              # body sprays, oils, deodorants, gift sets


df["product_type"] = df["Catatory"].apply(group_category)
print("Raw labels:", df["Catatory"].nunique(), "-> groups:", df["product_type"].nunique())
print(df["product_type"].value_counts())
log_step("Collapse category labels into 6 product types", df["Catatory"].notna().sum(), "Case, spelling and wording variants")

# 3.6 Brand: tidy whitespace; a missing brand is treated as 'Unbranded'
df["brand"] = df["Item_Name"].fillna("Unbranded").str.strip()
log_step("Missing brand set to 'Unbranded'", df["Item_Name"].isna().sum(), "Single row")

# 3.7 Location: keep the country (last part of 'City, State, Country')
df["country"] = df["itemLocation"].str.split(",").str[-1].str.strip()
df["ships_from_us"] = (df["country"] == "United States").astype(int)

# 3.8 Last update: parse dates (two month formats) and turn them into 'days since update'
dates = pd.to_datetime(df["lastUpdated"].str.replace(r" P[DS]T", "", regex=True), format="mixed", errors="coerce")
df["days_since_update"] = (dates.max() - dates).dt.total_seconds() / 86400
print("Unparsed or missing dates:", dates.isna().sum(), "| latest update:", dates.max())

# 3.9 Bottle size in ounces, parsed from the description (used for profiling, not clustering)
size = df["details"].str.extract(r"(\d+(?:\.\d+)?)\s*(?:fl\.?\s*)?oz", flags=re.IGNORECASE)[0].astype(float)
df["size_oz"] = size.where(size.between(0.3, 8.5))   # ignore implausible parses
print("Bottle size found for", df["size_oz"].notna().sum(), "listings")
```

Figure 9. Code screenshot: Normalisation of 63 category labels into six product types.
Source: COM7022 - Assignment/output/screenshots/code06_product_type.png

2.3 Transformation, Scaling and Categorical Variables

Logarithmic transformation. A log(1 + x) transform was applied to price and units sold. It preserves zero values and rank order while compressing the long right tail, reducing the skewness of units sold from 9.72 to 0.52 and of price from 1.97 to 0.09 (Table 3, Figure 10).

Standardisation. Because all three algorithms rely on Euclidean distance, each feature was converted to a z-score so that it contributes equal variance:

$$z = \frac{x - \mu}{\sigma}$$

Min-max normalisation was evaluated as an alternative and rejected: it inflates the bounded 1–11 stock scale relative to the log-transformed measures and changed the partition substantially (ARI 0.49; Section 4.5).

Categorical variables. Brand (246 values), product type and country were deliberately excluded from the distance computation. One-hot encoding would add hundreds of sparse dimensions and allow single brands to dominate the geometry; these attributes were instead reserved for post-hoc cluster profiling.

Table 3. Descriptive statistics of the 997 cleaned listings before and after the log transform.

Variable	Mean	Std	Min	Median	Max	Skewness
price (US$)	46.07	34.75	3	35.62	259.09	1.97
units sold	771.63	3203.77	0	49	54,052	9.72
stock level	7.73	3.53	1	9	11	-0.57
log price	3.63	0.67	1.39	3.6	5.56	0.09
log units sold	4.29	2.08	0	3.91	10.9	0.52

Source: COM7022 - Assignment/output/tables/table04_descriptive_statistics_clean.csv

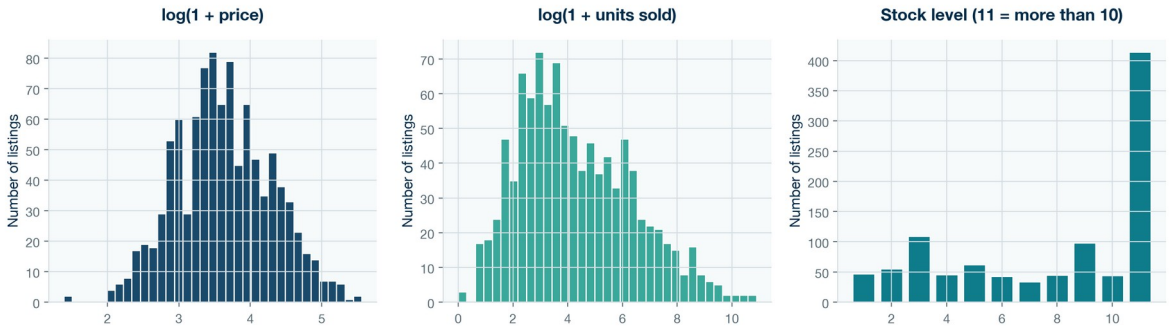


Figure 10. Feature distributions after the logarithmic transformation.
Source: COM7022 - Assignment/output/figures/fig03_transformed_distributions.png

COM7022_Clustering_25199053.ipynb | Transformation: log(1 + x)

```
# 3.10 Transform the skewed measures. log(1 + x) keeps zero sales valid (log of 0 is undefined).
df["log_price"] = np.log1p(df["price"])
df["log_sold"] = np.log1p(df["sold"])

skew_table = pd.DataFrame({
    "skew_before": [skew(df["price"]), skew(df["sold"])],
    "skew_after_log": [skew(df["log_price"]), skew(df["log_sold"])],
}, index=["price", "sold"]).round(2)
print(skew_table)

# Outliers: count them with the 1.5 x IQR rule on the raw scale, but keep them.
# Best-selling listings are exactly what the business wants to find, so removing them would be wrong.
q1, q3 = df["sold"].quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)
print("Listings above the sold upper fence (" + str(round(upper_fence)) + "):", (df["sold"] > upper_fence).sum())
log_step("Log-transform price and sold", len(df), "Outliers kept; log scale limits their pull on distances")

cleaning_table = pd.DataFrame(cleaning_log)
save_table(cleaning_table, "table03_cleaning_log.csv")
print("Rows after cleaning:", len(df))
cleaning_table
```

Figure 11. Code screenshot: Logarithmic transformation of skewed measures.
Source: COM7022 - Assignment/output/screenshots/code07_log_transform.png

COM7022_Clustering_25199053.ipynb | Scaling: StandardScal

```
FEATURES = ["log_price", "log_sold", "stock_level"]
scaler = StandardScaler()
X = scaler.fit_transform(df[FEATURES])
print("Feature matrix:", X.shape)
print("Means after scaling:", X.mean(axis=0).round(3))
print("Std after scaling:", X.std(axis=0).round(3))
```

Figure 12. Code screenshot: Z-score standardisation of the selected features.
Source: COM7022 - Assignment/output/screenshots/code08_scaling.png

2.4 Feature Selection

Candidate features were screened with Spearman rank correlation (Figure 13) and a systematic comparison of five feature sets (Table 4). The correlations are weak to moderate (price–sales −0.13, price–stock −0.20, sales–stock +0.35), confirming that each candidate captures a distinct commercial mechanism without problematic collinearity.

The final set comprises **log price, log units sold and stock level** (Set B), representing market position, demand and supply resilience. A two-feature baseline (Set A) achieves a marginally higher silhouette but ignores availability, which is central to the brief and to stock-out risk. Adding listing recency or bottle size reduced separation without adding actionable meaning: half of all bottles fall between 3.3 and 3.7 oz, and three-quarters of listings were updated within the past week.

Table 4. K-Means silhouette scores for five candidate feature sets (higher is better).

Feature set	Features	k = 2	k = 3	k = 4	k = 5	k = 6
A: price + sold	2	0.351	0.377	0.356	0.351	0.354
B: price + sold + stock	3	0.337	0.297	0.307	0.308	0.297
C: B + days since update	4	0.31	0.315	0.277	0.288	0.284
D: B + bottle size	4	0.274	0.308	0.271	0.272	0.272
E: B + days + size	5	0.256	0.259	0.292	0.255	0.258

Source: COM7022 - Assignment/output/tables/table05_feature_set_scan.csv

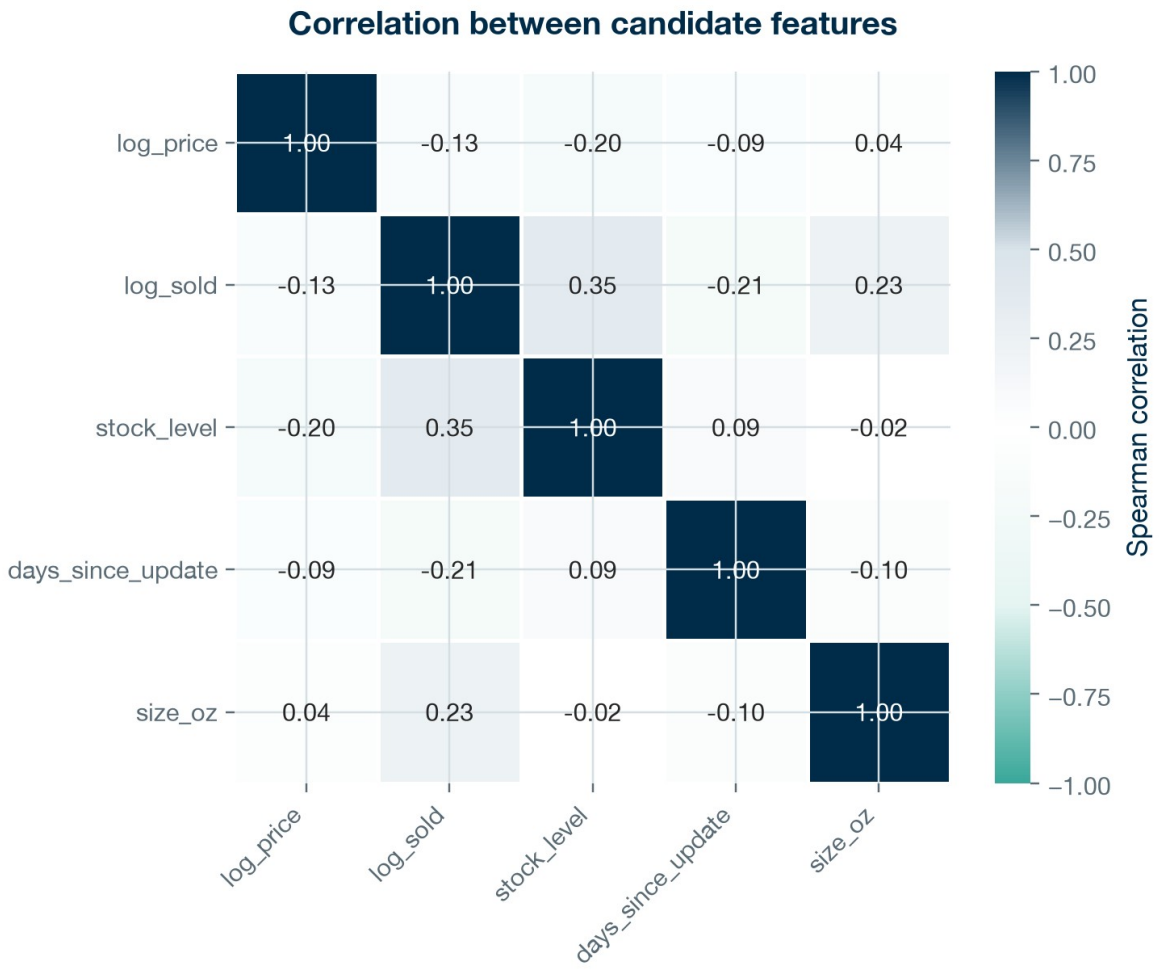


Figure 13. Spearman rank correlation between candidate features.
Source: COM7022 - Assignment/output/figures/fig04_feature_correlation.png

3. Model Selection

3.1 Clustering in Brief

Clustering is an **unsupervised learning** task: it partitions observations so that members of a group are more similar to one another than to members of other groups, without reference to known outcomes. It is the appropriate paradigm here because the catalogue carries no commercial labels; the objective is to reveal structure, not to predict a target.

3.2 Candidate Techniques

Three algorithmic families were selected because they embody different definitions of a "cluster" (Table 5):

- **K-Means** (MacQueen, 1967) is centroid-based. It minimises within-cluster variance by alternately assigning points to the nearest centroid and updating the centroids:

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

- **Agglomerative hierarchical clustering** builds a nested tree by repeatedly merging the closest clusters. Ward linkage (Ward, 1963) merges the pair that least increases total within-cluster variance.
- **DBSCAN** (Ester et al., 1996) is density-based. It grows clusters from dense neighbourhoods defined by a radius ε and a minimum point count, and labels isolated points as noise.

Table 5. Comparison of the three clustering paradigms applied.

Technique	Principle	Strengths	Limitations
K-Means	Centroid-based: minimises within-cluster variance	Fast, scalable, full coverage, interpretable centroids	Requires k; assumes compact, similar-sized clusters
Hierarchical (Ward)	Agglomerative: merges clusters into a dendrogram	No k needed upfront; reveals nested structure	Irreversible merges; $O(n^2)$ memory; sensitive to sampling
DBSCAN	Density-based: grows clusters from dense regions	Arbitrary shapes; explicit noise and outlier detection	Needs density gaps; sensitive to ε and MinPts

3.3 Dimensionality Reduction with PCA

Principal Component Analysis (Pearson, 1901) was applied to the standardised features. With only three inputs, PCA is not required to counter the curse of dimensionality; its role is to provide a faithful two-dimensional plane for visualising and interpreting clusters. The first two components retain **78.7%** of total variance (Figure 14, Table 6). **PC1 (48.6%)** loads positively on sales (0.64) and stock (0.65) and negatively on price (−0.42): it measures *commercial traction*. **PC2 (30.1%)** is dominated by price (0.91): it measures *price tier*. The PCA plane is therefore directly interpretable in business terms.

Table 6. PCA loadings of the standardised features.

Feature	PC1 (48.6%)	PC2 (30.1%)	PC3 (21.3%)
log_price	-0.421	0.906	0.051
log_sold	0.636	0.335	-0.695
stock_level	0.647	0.26	0.717

Source: COM7022 - Assignment/output/tables/table06_pca_loadings.csv



Figure 14. Explained variance (left) and loading biplot (right) of the principal components.
Source: COM7022 - Assignment/output/figures/fig05_pca_variance_biplot.png

4. Model Implementation and Evaluation

4.1 K-Means

K-Means was fitted with k-means++ initialisation, 20 restarts and a fixed seed (42) for k = 2 to 10 (Table 7, Figure 15). Inertia falls by 38.9% between k = 2 and k = 4 and flattens thereafter, locating the elbow. Although the silhouette peaks at k = 2 (0.337), a two-way split merely separates high from low volume. **k = 4** achieves the lowest Davies-Bouldin index of all candidates (1.015) with a solid silhouette (0.307), and yields commercially viable segments of 22–31% of listings. All four clusters have positive mean silhouette widths (0.29–0.32) and only 1.0% of listings carry a negative value (Figure 16).

Table 7. K-Means model scan for k = 2 to 10.

k	Inertia	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Smallest cluster
2	1943	0.337	1.211	536.7	393
3	1501	0.297	1.228	493.4	298
4	**1186.7**	**0.307**	**1.015**	**503.3**	**219**
5	974.4	0.308	1.018	513.3	156
6	862.5	0.297	1.053	489.1	141
7	766.7	0.307	1.043	478.7	84
8	691.8	0.296	1.057	469.6	82
9	640.9	0.296	1.029	452.9	59
10	594.6	0.294	1.037	442	53

Source: COM7022 - Assignment/output/tables/table07_kmeans_k_scan.csv



Figure 15. Elbow (inertia) and silhouette criteria across k.
Source: COM7022 - Assignment/output/figures/fig06_kmeans_elbow_silhouette.png

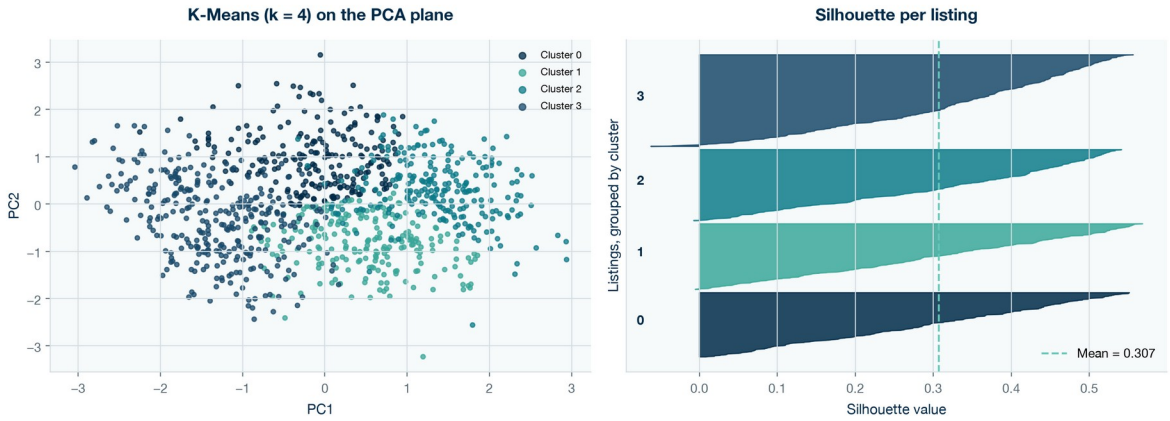


Figure 16. K-Means (k = 4) on the PCA plane and per-cluster silhouette profile.
Source: COM7022 - Assignment/output/figures/fig07_kmeans_clusters_silhouette.png

4.2 Hierarchical Clustering

Four linkage criteria were compared on identical Euclidean distances (Table 8). Single and average linkage suffer from *chaining*, absorbing 994 and 989 listings respectively into one cluster; complete linkage produces uneven groups (424/388/180/5). **Ward linkage** yields balanced, interpretable groups (358/237/235/167) and the highest silhouette among linkages (0.253). Its dendrogram separates into four branches at a merge height of roughly 21 (Figure 17), supporting a four-cluster cut that also enables a like-for-like comparison with K-Means. Agreement between the two partitions is moderate (ARI 0.41).

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2$$

Table 8. Comparison of four hierarchical linkage criteria.

Linkage	Cophenetic correlation	Silhouette (k = 4)	Cluster sizes
single	0.362	0.217	994/1/1/1
complete	0.536	0.197	424/388/180/5
average	0.631	0.206	989/6/1/1
ward	0.579	0.253	358/237/235/167

Source: COM7022 - Assignment/output/tables/table08_linkage_comparison.csv

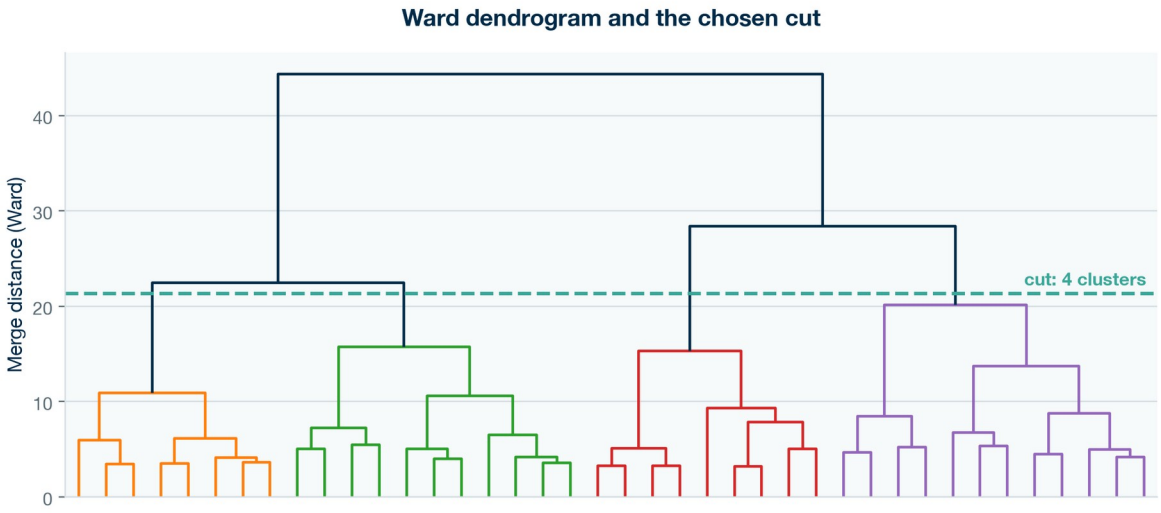


Figure 17. Ward dendrogram with the four-cluster cut.
Source: COM7022 - Assignment/output/figures/fig09_ward_dendrogram_cut.png

4.3 DBSCAN

MinPts was set to 6 (twice the dimensionality) and ε was read from the sorted 6-nearest-neighbour distance curve (Figure 18), followed by a 32-configuration grid search. The curve rises smoothly without a clear knee — the first sign that the catalogue lacks dense, separated regions. Small radii (ε ≤ 0.35) fragment the data and reject 13–89% of listings as noise; larger radii (ε ≥ 0.5) merge almost everything into one cluster. The best compromise (**ε = 0.45**) places 904 listings (90.7%) in one cluster, 4 in a second and 89 (8.9%) in noise, with a slightly negative silhouette (−0.014) (Figure 19).

This is an informative negative result: the listings form a **continuum** of price and performance rather than density islands, so DBSCAN cannot segment them. Its noise set nevertheless has a distinct profile — lower prices (median $26 vs $36), higher sales (163 vs 46 units) and thinner stock (5 vs 9) — which makes DBSCAN useful as an anomaly detector.

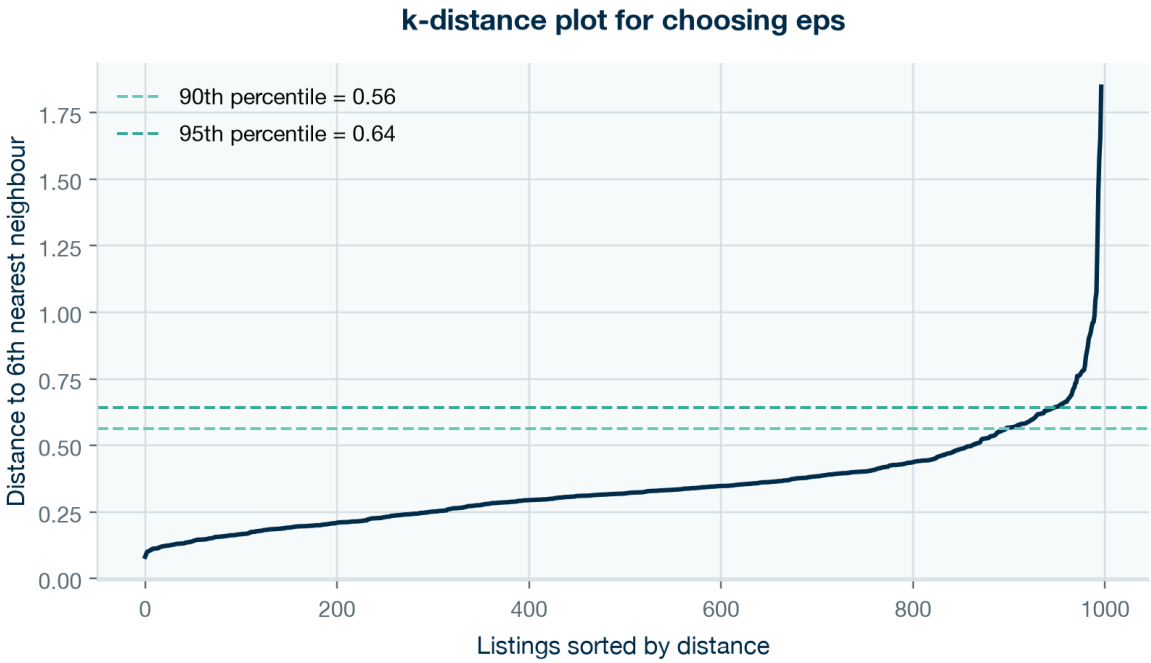


Figure 18. Sorted 6-nearest-neighbour distances used to select ε.
Source: COM7022 - Assignment/output/figures/fig10_dbscan_k_distance.png

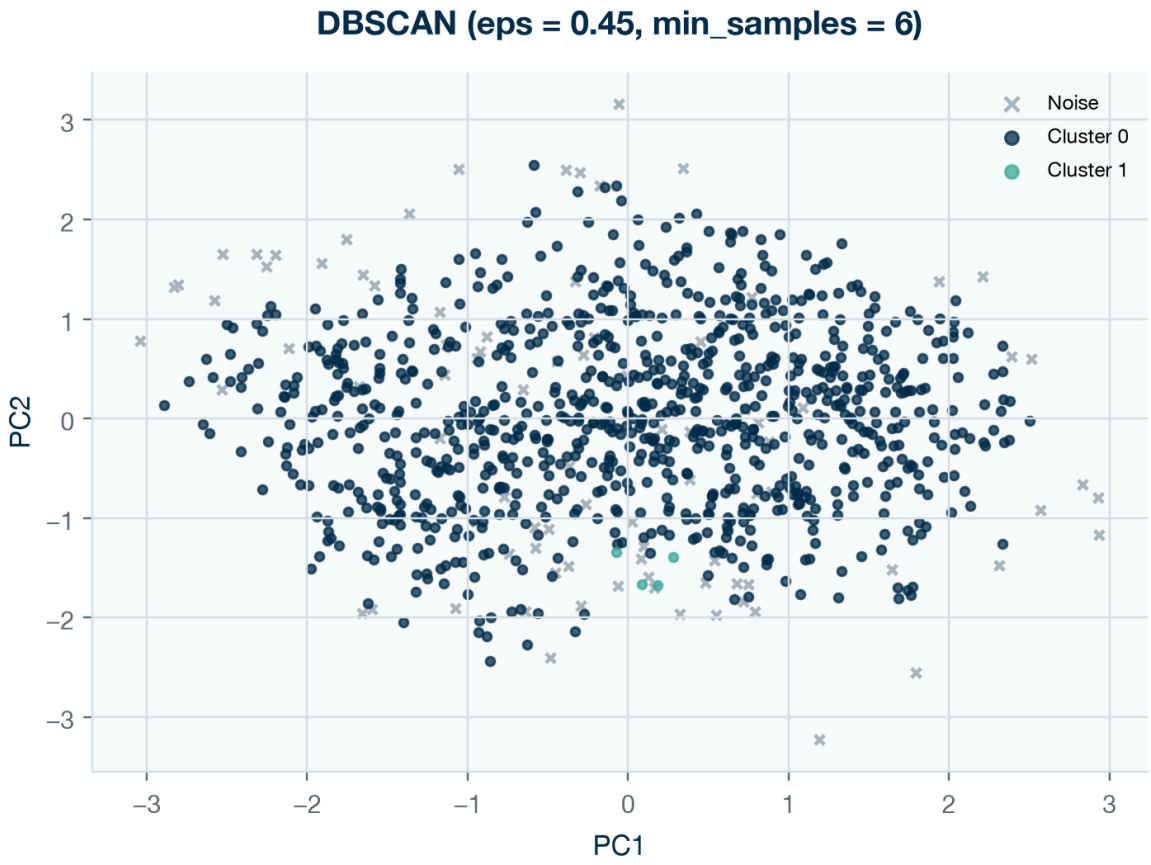


Figure 19. DBSCAN partition on the PCA plane; grey points are noise.
Source: COM7022 - Assignment/output/figures/fig11_dbscan_clusters.png

4.4 Evaluation Framework

Accuracy-type metrics require ground-truth labels and are therefore inapplicable to clustering. Model quality was instead triangulated across complementary measures, each answering a different question (Table 9). The silhouette coefficient (Rousseeuw, 1987) is defined for each listing as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}}$$

where a(i) is the mean distance to its own cluster and b(i) to the nearest other cluster. Davies-Bouldin (Davies and Bouldin, 1979) and Calinski-Harabasz (Caliński and Harabasz, 1974) capture compactness relative to separation, while the Adjusted Rand Index (Hubert and Arabie, 1985) measures agreement between partitions and underpins the stability tests.

Table 9. Evaluation framework: complementary measures and what each can and cannot tell us.

Measure	What it captures	Better	Caveat
Silhouette	Cohesion vs separation of each listing (−1 to 1)	Higher	Favours convex clusters; ignores stability
Davies-Bouldin	Similarity of each cluster to its closest rival	Lower	Assumes centroid-shaped clusters
Calinski-Harabasz	Between- to within-cluster dispersion ratio	Higher	Inflated for convex clusters; weak for DBSCAN
Bootstrap stability (ARI)	Agreement of labels over 30 resamples of 80%	Higher (max 1)	Measures reproducibility, not quality
Sensitivity (ARI)	Agreement after changing seed, scaling or outliers	Higher (max 1)	Tests robustness to analyst choices
Coverage and balance	Share of noise; smallest and largest cluster	Low noise, balanced	Business usability rather than geometry

4.5 Comparative Results

Table 10. Comparative performance of the three final models.

Model	Clusters	Noise %	Silhouette ↑	DBI ↓	CH ↑	Stability ARI ↑	Smallest %	Largest %
K-Means	**4**	**0**	**0.307**	**1.015**	**503.3**	**0.932 ± 0.025**	**22**	**31.3**
Hierarchical (Ward)	4	0	0.253	1.257	402.9	0.481 ± 0.084	16.8	35.9
DBSCAN	2	8.9	-0.014	1.172	3.5	0.749 ± 0.063	0.4	90.7

Source: COM7022 - Assignment/output/tables/table11_model_comparison.csv

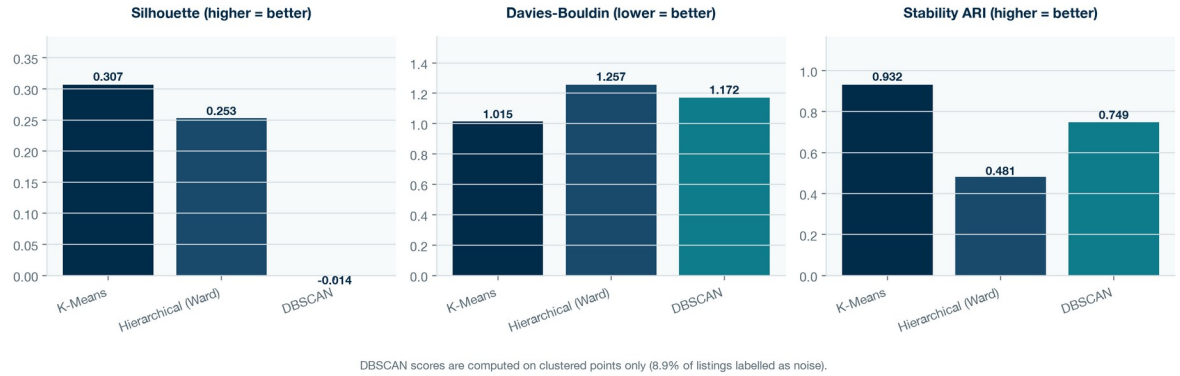


Figure 20. Side-by-side comparison of internal validity and stability.
Source: COM7022 - Assignment/output/figures/fig12_model_comparison.png

K-Means leads on **every internal index** — silhouette 0.307 versus 0.253 (Ward) and −0.014 (DBSCAN), Davies-Bouldin 1.015 versus 1.257 and 1.172, Calinski-Harabasz 503.3 versus 402.9 and 3.5. Stability is decisive: across 30 resamples of 80% of the data, K-Means reproduces its partition almost exactly (**ARI 0.932 ± 0.025**), whereas Ward does not (0.481 ± 0.084) because early merges in a hierarchy are irreversible and small sample changes propagate up the tree.

Sensitivity tests confirm robustness to analytical choices (Table 11): different random seeds (mean ARI 0.984, minimum 0.951) and removal of the top 1% of sellers (0.937) barely alter the solution, which also shows that the log transform neutralised outlier leverage. Only the switch to min-max scaling (0.492) materially changes the partition, justifying the standardisation decision.

Table 11. Robustness of the K-Means solution to analytical choices.

Sensitivity test (K-Means, k = 4)	ARI
K-Means across 10 seeds (mean ARI)	0.984
z-score vs min-max scaling (ARI)	0.492
with vs without top 1% sellers (ARI)	0.937

Source: COM7022 - Assignment/output/tables/table12_sensitivity_checks.csv

> **Model Selection Decision: K-Means (k = 4)** is selected as the production segmentation engine: best internal validity, near-perfect stability and full catalogue coverage. Ward is retained as a structural cross-check and DBSCAN as an operational anomaly flag.

5. Cluster Characteristics

Mapping centroids back to business units reveals four distinct commercial segments (Table 12, Figure 21, Figure 22):

- **Cluster 2 — Best-Sellers (242 listings, 24.3%).** The revenue engine: **92.3% of all units sold** (710,124 of 769,317), median 952 units at an accessible $29.94, full stock and 99.6% US dispatch. Predominantly Eau de Toilette from high-demand lines such as Armaf and Ralph Lauren.
- **Cluster 0 — Premium Designer (219 listings, 22.0%).** The prestige tier: highest prices (median $68.99, up to $259.09) with steady but slow demand (median 43 units, 3.7% of volume). Concentrated Eau de Parfum from Giorgio Armani, Paco Rabanne and Yves Saint Laurent.
- **Cluster 1 — Budget Commodities (224 listings, 22.5%).** The cheapest offers (median $18.99) yet weak traction (median 30 units, 1.8% of volume), with the highest share of unbranded and generic items: low price alone does not create demand without brand equity.

- **Cluster 3 — Low-Stock Tail (312 listings, 31.3%).** The largest segment: median stock of only 3 units, 14.1% showing "Last One" and minimal sales (median 16 units, 2.2% of volume) — an imminent stock-out and churn risk.

Table 12. Business profile of the four K-Means segments.

Measure	C2 Best-Sellers	C0 Premium	C1 Budget	C3 Low-Stock
Listings	242	219	224	312
Share of listings (%)	24.3	22	22.5	31.3
Median price (US$)	29.94	68.99	18.99	39.99
Median units sold	952	43	30	16
Share of units sold (%)	92.3	3.7	1.8	2.2
Median stock level	11	11	11	3
Listings at 'Last One' (%)	1.2	0	0	14.1
Ships from US (%)	99.6	84.9	92	88.5
Dominant product type	Eau de Toilette (67.8%)	Eau de Parfum (45.2%)	Eau de Toilette (37.9%)	Eau de Toilette (39.1%)

Source: COM7022 - Assignment/output/tables/table13_cluster_profiles.csv

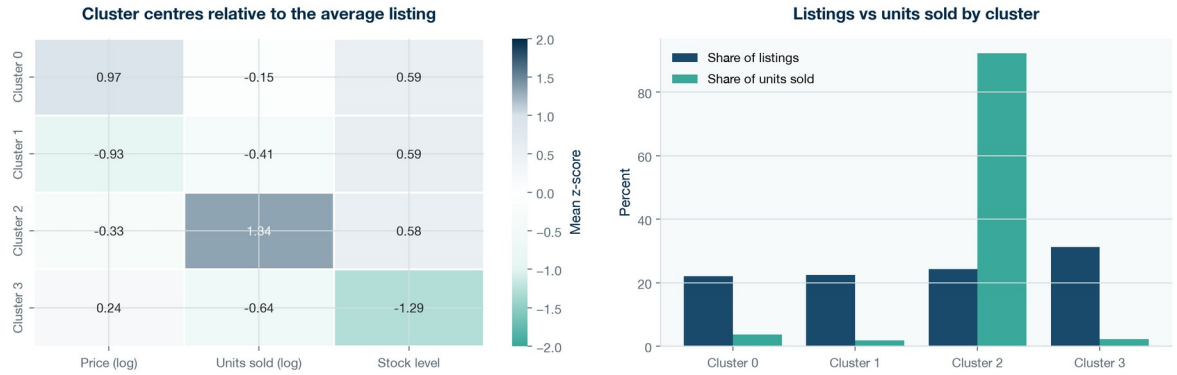


Figure 21. Standardised feature means per cluster.
Source: COM7022 - Assignment/output/figures/fig13_cluster_profiles.png

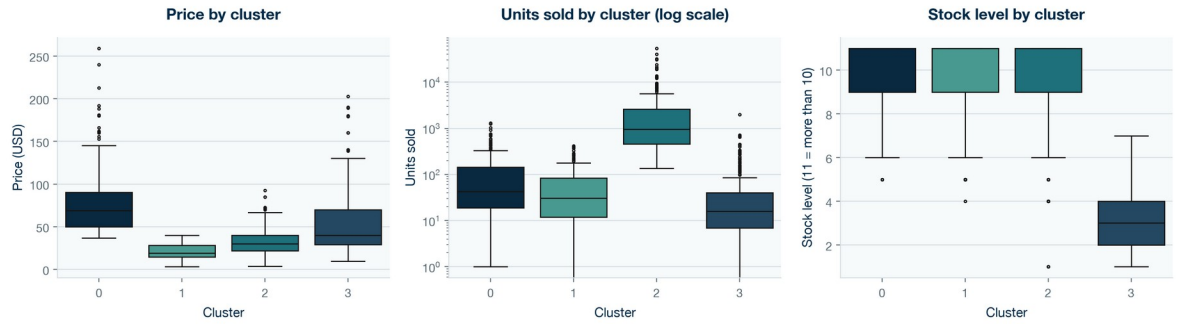


Figure 22. Price, sales and stock distributions within each cluster.
Source: COM7022 - Assignment/output/figures/fig14_cluster_distributions.png

TASK 2

Critical Report for Executive Stakeholders

834 words of body text

6. Executive Summary

Machine learning reveals that our 997 fragrance listings fall into **four natural commercial segments**. One quarter of listings — the best-sellers — generate **92% of all units sold**, while almost a third of the catalogue is close to running out of stock. The recommended model, **K-Means with four segments**, is the most accurate and stable of the three techniques tested and is simple enough to run automatically on new listings. Each segment calls for a distinct action: protect, grow, reposition or audit.

CORE REVENUE ENGINE	INVENTORY RISK	MODEL STABILITY	PCA VARIANCE
92.3%	**31.3%**	**0.932**	**78.7%**
of units sold come from 24.3% of listings (Cluster 2)	of listings sit in the low-stock tail (Cluster 3)	K-Means bootstrap ARI over 30 resamples	retained by the first two components

7. Critical Evaluation

7.1 Suitability of the Methods and Design Decisions

Unsupervised clustering is the right tool because the company holds no labelled outcomes; the aim is discovery rather than prediction. Three design decisions shaped the outcome. The log transformation stopped a handful of viral listings from dictating the segmentation. The choice of price, sales and stock answers the three questions managers actually ask — *what is it worth, does it sell, will it run out?* Excluding brand and country from the distance calculation prevented segments from degenerating into brand lists. Each decision was tested rather than assumed: the result survives changes of seed and the removal of top sellers.

7.2 Comparative Performance: Strengths and Limitations

K-Means was the strongest model on every measure. Its limitation is that it assumes compact, similarly sized groups and requires the number of segments to be chosen; on a continuous catalogue its boundaries are therefore soft, as the moderate silhouette (0.31) honestly reflects. **Ward hierarchical clustering** offers an intuitive tree of nested groups but proved unstable: its segments changed substantially whenever the sample changed, and it cannot assign new listings without rebuilding the tree. **DBSCAN** excels at irregular shapes and outlier detection, but because our listings form a continuum it placed 91% of them in one group and left 9% unassigned. These weaknesses are properties of the data as much as of the algorithms, which is itself a valuable insight into how the marketplace is structured.

7.3 Why K-Means Is the Right Choice for E-Commerce

Technical scores are only half the decision. An e-commerce platform needs a model that covers every listing, can be refreshed frequently, scores new listings instantly, scales to millions of offers and can be explained to managers (Table 13). K-Means satisfies all five requirements: each new listing is assigned to its nearest centroid in microseconds, and each centroid reads as a typical price, sales level and stock position. A simple sales

ranking would not suffice either: it identifies who sells most, but not who is premium, who is cheap yet stagnant, or who is about to run out. Segmentation combines all three dimensions into groups that each require a different response.

Table 13. Operational suitability of each model for an e-commerce platform.

E-commerce requirement	K-Means	Ward hierarchical	DBSCAN
Covers every listing	Yes · 0% noise	Yes · 0% noise	No · 8.9% unassigned
Scores new listings instantly	Yes · nearest centroid	No · tree must be rebuilt	Partly · neighbourhood search
Stable between refreshes	Yes · ARI 0.93	No · ARI 0.48	Partly · ARI 0.75
Scales to millions of listings	Yes · linear	No · quadratic memory	Partly · index-dependent
Explainable to managers	Yes · four typical profiles	Partly · dendrogram	No · one dominant group
Recommended role	Production segmentation	Structural cross-check	Anomaly monitor

8. Recommendations for Business Managers

The four segments translate into targeted, measurable interventions (Figure 23):

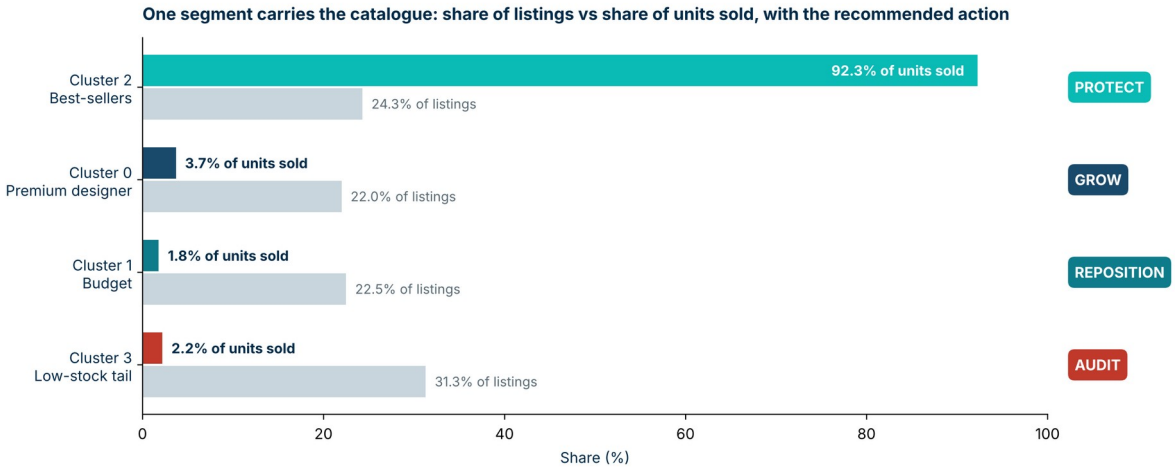


Figure 23. Share of listings versus share of units sold per segment, with the recommended action.
Source: COM7022 - Assignment/output/figures/fig17_segment_actions.png

PROTECT · CLUSTER 2

Secure the revenue engine

Evidence: 92.3% of units sold from 24.3% of listings.

Action: Automated restock alerts below 10 units, priority search placement and fulfilment SLAs for these sellers.

GROW · CLUSTER 0

Monetise the premium tier

Evidence: Highest prices (median $68.99) with steady, slow demand.

Action: Bundle with best-sellers on product pages and invest in premium content rather than discounting.

REPOSITION · CLUSTER 1

Stop subsidising weak demand

Evidence: Cheapest offers (median $18.99) yet only 1.8% of volume.

Action: End paid promotion of unbranded items and reallocate budget to proven brands.

AUDIT · CLUSTER 3

Clean up the low-stock tail

Evidence: 31.3% of listings, median stock of 3, 14.1% at 'Last One'.

Action: 30-day merchant restock notices; delist listings that remain empty and unsold after 60 days.

Implementation should begin with the best-sellers and the low-stock tail, which protect the most revenue for the least effort. Success should be tracked through stock-outs in Cluster 2, the proportion of Cluster 3 listings restocked or delisted within 30 days, and sales per advertising dollar in Cluster 1. The segmentation should be refreshed monthly, with new listings scored on arrival and DBSCAN noise points reviewed as potential anomalies.

9. Limitations

The platform caps visible stock at ten units, so genuine depth above that level is unknown. The data is a single snapshot of one product family, so segment boundaries may differ in other categories or seasons. Clusters describe association, not causation: a premium price does not itself make a listing slow. Finally, because listings form a continuum, those near a boundary could reasonably belong to either neighbouring segment; the segments should guide prioritisation rather than replace commercial judgement.

10. Conclusion and Future Work

This project demonstrates that a carefully prepared, well-validated K-Means model can turn an unlabelled and messy catalogue into four clear, actionable segments, and that the commercial value of the catalogue is extraordinarily concentrated. The most important lesson is that **data preparation mattered more than algorithm choice**: recovering values from listing text and correcting the hidden stock ceiling changed the outcome more than any hyperparameter. Equally, DBSCAN's failure was not wasted effort; it provided the clearest evidence that the marketplace is a continuum rather than a set of discrete niches.

Future work would enrich the feature space with revenue, margin and true stock counts, incorporate text embeddings of product descriptions, and track segment migration over time to detect listings that are rising or declining. Validating whether segment membership predicts next-quarter sales would convert the descriptive model into a forward-looking decision tool, and a lightweight dashboard would place each listing's segment directly in the hands of category managers.

References

Caliński, T. and Harabasz, J. (1974) 'A dendrite method for cluster analysis', *Communications in Statistics*, 3(1), pp. 1–27.

Davies, D.L. and Bouldin, D.W. (1979) 'A cluster separation measure', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), pp. 224–227.

Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996) 'A density-based algorithm for discovering clusters in large spatial databases with noise', in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. Portland: AAAI Press, pp. 226–231.

Hubert, L. and Arabie, P. (1985) 'Comparing partitions', *Journal of Classification*, 2(1), pp. 193–218.

MacQueen, J. (1967) 'Some methods for classification and analysis of multivariate observations', in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Berkeley: University of California Press, pp. 281–297.

McKinney, W. (2010) 'Data structures for statistical computing in Python', in *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

Müller, A.C. and Guido, S. (2016) *Introduction to Machine Learning with Python*. Sebastopol: O'Reilly Media.

Pearson, K. (1901) 'On lines and planes of closest fit to systems of points in space', *Philosophical Magazine*, 2(11), pp. 559–572.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Rousseeuw, P.J. (1987) 'Silhouettes: a graphical aid to the interpretation and validation of cluster analysis', *Journal of Computational and Applied Mathematics*, 20, pp. 53–65.

Ward, J.H. (1963) 'Hierarchical grouping to optimize an objective function', *Journal of the American Statistical Association*, 58(301), pp. 236–244.

Appendix A: Reproducible Code

The complete Task 1 implementation is a single Jupyter notebook that executes end to end in about 30 seconds on Google Colab or locally, with fixed random seeds (42). All materials — notebook, dataset, figures, tables and code screenshots — are hosted in the project repository: https://github.com/kvmmn/_COM7022_MachineLearning. The full notebook code follows, one screenshot per cell in execution order.

A · 0. Set-up

COM7022_Clustering_25199053.ipynb | code cell 1 (notebook cell 2)

```
# Standard libraries
import os
import re
import json
import warnings
from pathlib import Path

# Data handling and plotting
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
import sklearn

# Machine learning tools
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (silhouette_score, silhouette_samples, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score)
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster, cophenet
from scipy.spatial.distance import pdist
from scipy.stats import skew

warnings.filterwarnings("ignore", category=FutureWarning)

# One seed for every random step, so a re-run gives the same numbers
SEED = 42
np.random.seed(SEED)

print("Python libraries used in this run:")
print("pandas", pd.__version__, "| numpy", np.__version__, "| scikit-learn", sklearn.__version__,
      "| scipy", scipy.__version__, "| matplotlib", matplotlib.__version__, "| seaborn", sns.__version__)
```

Code A1. Notebook code cell 1
Source: COM7022 - Assignment/output/screenshots/appendix/cell01.png

COM7022_Clustering_25199053.ipynb | code cell 2 (notebook cell 3)

```
# Arden University palette (Midnight / Tiffany), the same visual system used in my earlier modules
MIDNIGHT = "#002B49"
TIFFANY = "#66C9BA"
TIFFANY_DEEP = "#3AA89A"
MIDNIGHT_SOFT = "#1A4A6B"
TEAL = "#0E7C8B"
MIST = "#E6F5F2"
PAPER = "#F7FAFB"
MUTE = "#5A6B75"
GRID = "#D5DEE3"
NOISE_GREY = "#A7B4BC"

# Cluster colours in a fixed order (five at most, then the chart should be split)
CLUSTER_COLOURS = [MIDNIGHT, TIFFANY_DEEP, TEAL, MIDNIGHT_SOFT, TIFFANY]
ARDEN_CMAP = matplotlib.colors.LinearSegmentedColormap.from_list(
    "arden", [MIST, TIFFANY, TIFFANY_DEEP, MIDNIGHT_SOFT, MIDNIGHT])
ARDEN_DIVERGING = matplotlib.colors.LinearSegmentedColormap.from_list(
    "arden_div", [TIFFANY_DEEP, MIST, "#FFFFFF", "#D9E3EA", MIDNIGHT])

plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["Aptos", "Helvetica Neue", "Arial", "DejaVu Sans"],
    "figure.facecolor": "white",
    "axes.facecolor": PAPER,
    "axes.edgecolor": GRID,
    "axes.labelcolor": MIDNIGHT,
    "axes.titlecolor": MIDNIGHT,
    "axes.titleweight": "bold",
    "axes.titlesize": 13,
    "axes.titlepad": 15,
    "axes.labelsize": 11,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.color": GRID,
    "xtick.color": MUTE,
    "ytick.color": MUTE,
    "legend.frameon": False,
    "figure.dpi": 110,
    "savefig.dpi": 220,
    "savefig.bbox": "tight",
})

# Output folders for the figures and tables used in the report
OUTPUT_DIR = Path("output")
FIG_DIR = OUTPUT_DIR / "figures"
TABLE_DIR = OUTPUT_DIR / "tables"
FIG_DIR.mkdir(parents=True, exist_ok=True)
TABLE_DIR.mkdir(parents=True, exist_ok=True)


def save_figure(fig, filename):
    # Save every chart with the same settings so the report pack is consistent
    fig.savefig(FIG_DIR / filename)
    print("Saved:", FIG_DIR / filename)


def save_table(table, filename):
    table.to_csv(TABLE_DIR / filename)
    print("Saved:", TABLE_DIR / filename)
```

Code A2. Notebook code cell 2

Source: COM7022 - Assignment/output/screenshots/appendix/cell02.png

A · 1. Data import

COM7022_Clustering_25199053.ipynb | code cell 3 (notebook cell 5)

```
DATA_FILE = "Sales-[5444].csv"
candidate_paths = [Path(DATA_FILE), Path("..") / DATA_FILE, Path("/content") / DATA_FILE]

data_path = None
for path in candidate_paths:
    if path.exists():
        data_path = path
        break

if data_path is None:
    try:
        from google.colab import files  # only available on Colab
        print("CSV not found. Please upload", DATA_FILE)
        uploaded = files.upload()
        data_path = Path(list(uploaded.keys())[0])
    except ImportError:
        raise FileNotFoundError("Place " + DATA_FILE + " next to this notebook.")

raw = pd.read_csv(data_path)
print("Loaded:", data_path)
print("Shape:", raw.shape[0], "rows x", raw.shape[1], "columns")
raw.head()
```

Code A3. Notebook code cell 3

Source: COM7022 - Assignment/output/screenshots/appendix/cell03.png

COM7022_Clustering_25199053.ipynb | c

```
# Column types and non-null counts
raw.info()
```

Code A4. Notebook code cell 4

Source: COM7022 - Assignment/output/screenshots/appendix/cell04.png

A · 2. Descriptive statistics and data quality

COM7022_Clustering_25199053.ipynb | code cell 5 (notebook cell 9)

```
# Missing values per column, as counts and percentages
missing = pd.DataFrame({
    "missing_count": raw.isna().sum(),
    "missing_pct": (raw.isna().mean() * 100).round(1),
})
missing = missing.sort_values("missing_count", ascending=False)
print("Full-row duplicates:", raw.duplicated().sum())
print("Duplicate listing descriptions:", raw["details"].duplicated().sum())
save_table(missing, "table01_missing_values.csv")
missing
```

Code A5. Notebook code cell 5
Source: COM7022 - Assignment/output/screenshots/appendix/cell05.png

COM7022_Clustering_25199053.ipynb | code cell 6 (notebook cell 10)

```
# Figure 1: where the gaps are
fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.barh(missing.index[::-1], missing["missing_count"][::-1], color=MIDNIGHT_SOFT)
# Highlight the biggest gap, since it drives a pre-processing decision
bars[-1].set_color(TIFFANY_DEEP)
for bar, value in zip(bars, missing["missing_count"][::-1]):
    ax.text(bar.get_width() + 1.5, bar.get_y() + bar.get_height() / 2, str(value),
            va="center", color=MIDNIGHT, fontsize=9)
ax.set_title("Missing values per column (n = 1,000 listings)")
ax.set_xlabel("Number of missing values")
ax.grid(axis="y", visible=False)
save_figure(fig, "fig01_missing_values.png")
plt.show()
```

Code A6. Notebook code cell 6
Source: COM7022 - Assignment/output/screenshots/appendix/cell06.png

COM7022_Clustering_25199053.ipynb | code cell 7 (notebook cell 11)

```
# Descriptive statistics for the numeric columns, including skewness
numeric_cols = ["price", "available", "sold"]
desc = raw[numeric_cols].describe().T
desc["median"] = raw[numeric_cols].median()
desc["skewness"] = [skew(raw[c].dropna()) for c in numeric_cols]
desc = desc[["count", "mean", "median", "std", "min", "25%", "75%", "max", "skewness"]].round(2)
save_table(desc, "table02_descriptive_statistics_raw.csv")
desc
```

Code A7. Notebook code cell 7
Source: COM7022 - Assignment/output/screenshots/appendix/cell07.png

COM7022_Clustering_25199053.ipynb | code cell 8 (notebook cell 12)

```
# Text and category columns: how many distinct values, and the most common ones
for col in ["Item_Name", "Catatory", "itemLocation"]:
    print(col, "->", raw[col].nunique(), "distinct values")
print()
print("Category labels (top 15 of", raw["Catatory"].nunique(), "):")
print(raw["Catatory"].value_counts().head(15))
```

Code A8. Notebook code cell 8
Source: COM7022 - Assignment/output/screenshots/appendix/cell08.png

COM7022_Clustering_25199053.ipynb | code cell 9 (notebook cell 13)

```
# The 'text' column holds the availability and sales wording shown on the listing.
# Replacing digits with N shows the small number of patterns behind 1,000 rows.
patterns = raw["text"].fillna("<missing>").str.split("/").str[0].str.strip()
patterns = patterns.str.replace(r"\d[\d,]*", "N", regex=True)
print(patterns.value_counts())
print()
print("Currencies in priceWithCurrency:")
print(raw["priceWithCurrency"].str.extract(r"^([A-Z]+ ?\$)")[0].value_counts())
```

Code A9. Notebook code cell 9
Source: COM7022 - Assignment/output/screenshots/appendix/cell09.png

COM7022_Clustering_25199053.ipynb | code cell 10 (notebook cell 15)

```
# Figure 2: raw distributions of price and units sold
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].hist(raw["price"].dropna(), bins=40, color=MIDNIGHT_SOFT, edgecolor="white")
axes[0].set_title("Price (raw)")
axes[0].set_xlabel("Price (USD)")
axes[0].set_ylabel("Number of listings")
axes[1].hist(raw["sold"].dropna(), bins=40, color=TIFFANY_DEEP, edgecolor="white")
axes[1].set_title("Units sold (raw)")
axes[1].set_xlabel("Units sold")
axes[1].set_ylabel("Number of listings")
fig.suptitle("Both measures are right-skewed; units sold is extreme", color=MIDNIGHT, fontweight="bold", y=1.03)
fig.tight_layout()
save_figure(fig, "fig02_raw_distributions.png")
plt.show()
```

Code A10. Notebook code cell 10

Source: COM7022 - Assignment/output/screenshots/appendix/cell10.png

A · 3. Pre-processing

COM7022_Clustering_25199053.ipynb | code cell 11 (notebook cell 17)

```
df = raw.copy()
cleaning_log = []


def log_step(step, rows_affected, note):
    cleaning_log.append({"step": step, "rows_affected": int(rows_affected), "note": note})


# 3.1 Price: recover missing prices from priceWithCurrency (same listing, same number)
price_from_text = df["priceWithCurrency"].str.extract(r"\$([\d,]+\.?\d*)")[0]
price_from_text = price_from_text.str.replace(",", "").astype(float)
recovered = df["price"].isna() & price_from_text.notna()
df.loc[recovered, "price"] = price_from_text[recovered]
log_step("Recover price from priceWithCurrency", recovered.sum(), "Exact same value, only stored in text form")

# Check: where both exist, do they agree?
both = df["price"].notna() & price_from_text.notna()
print("Price disagreements where both exist:", (abs(df.loc[both, "price"] - price_from_text[both]) > 0.01).sum())

# 3.2 Currency: convert the three Canadian-dollar prices to USD
# Approximate May 2024 rate (1 CAD ~ 0.73 USD). Only 3 rows, so the exact rate barely matters.
CAD_TO_USD = 0.73
is_cad = df["priceWithCurrency"].fillna("").str.startswith("C")
df.loc[is_cad, "price"] = (df.loc[is_cad, "price"] * CAD_TO_USD).round(2)
log_step("Convert CAD prices to USD", is_cad.sum(), "Fixed rate 0.73, documented approximation")
```

Code A11. Notebook code cell 11

Source: COM7022 - Assignment/output/screenshots/appendix/cell11.png

COM7022_Clustering_25199053.ipynb | code cell 12 (notebook cell 18)

```
# 3.3 Units sold: recover from the 'text' column ("... / 116 sold")
sold_from_text = df["text"].str.extract(r"([\d,]+) sold")[0].str.replace(",", "").astype(float)
recovered = df["sold"].isna() & sold_from_text.notna()
df.loc[recovered, "sold"] = sold_from_text[recovered]
log_step("Recover sold from text", recovered.sum(), "Exact value from the listing wording")

# Check the two sources agree everywhere else
both = df["sold"].notna() & sold_from_text.notna()
print("Sold disagreements where both exist:", (df.loc[both, "sold"] != sold_from_text[both]).sum())

# Listings whose text shows stock but no 'sold' part have not sold yet, so sold = 0
no_sales_yet = df["sold"].isna() & df["text"].notna() & ~df["text"].str.contains("sold", na=False)
df.loc[no_sales_yet, "sold"] = 0
log_step("Set sold = 0 where text shows stock but no sales", no_sales_yet.sum(), "Listing wording omits 'sold' when nothing has sold")

# Rows with no text and no sold value carry no sales or stock information at all
no_info = df["sold"].isna() & df["text"].isna()
print(df.loc[no_info, ["Item_Name", "price", "available", "text", "sold"]])
df = df[~no_info].copy()
log_step("Drop rows with no sales or stock information", no_info.sum(), "Cannot be placed on the sales-performance axis")
```

Code A12. Notebook code cell 12

Source: COM7022 - Assignment/output/screenshots/appendix/cell12.png

COM7022_Clustering_25199053.ipynb | code cell 13 (notebook cell 19)

```
# 3.4 Stock level: rebuild it from the listing wording so every row has a value.
# The marketplace hides exact numbers above 10 ("More than 10 available"), so I cap all
# stock values at 11, meaning "more than 10". This keeps the measure honest and comparable.
text = df["text"].fillna("")
stock = text.str.extract(r"^(\d+) available")[0].astype(float)
stock[text.str.startswith("More than")] = 11
stock[text.str.startswith("Last One")] = 1
capped_above_10 = (stock > 11).sum()
stock = stock.clip(upper=11)

# "Limited quantity available" and lot listings give no number: impute the median, but keep a flag
stock_imputed = stock.isna()
df["stock_imputed"] = stock_imputed.astype(int)
stock = stock.fillna(stock.median())
df["stock_level"] = stock

log_step("Stock: 'More than 10' coded as 11", text.str.startswith("More than").sum(), "Censored value, not exact")
log_step("Stock: explicit values above 10 capped at 11", capped_above_10, "Same scale as the censored rows")
log_step("Stock: 'Last One' coded as 1", text.str.startswith("Last One").sum(), "Wording means one unit left")
log_step("Stock: median imputation", stock_imputed.sum(), "Limited quantity / lot wording; flagged in stock_imputed")
print(df["stock_level"].value_counts().sort_index())
```

Code A13. Notebook code cell 13

Source: COM7022 - Assignment/output/screenshots/appendix/cell13.png

COM7022_Clustering_25199053.ipynb | code cell 14 (notebook cell 20)

```
# 3.5 Product type: collapse the messy category labels into six groups


def group_category(label):
    if pd.isna(label):
        return "Other"
    s = str(label).strip().lower()
    if s in ["", "/", "y", "does not apply", "unknown", "various", "assorted"]:
        return "Other"
    if "," in s or "/" in s:
        return "Other"                          # listings that name several types at once
    if "toil" in s or s == "edt":
        return "Eau de Toilette"                # includes the misspelling 'toillette'
    if "eau de parfum" in s or "eau de perfume" in s:
        return "Eau de Parfum"
    if "parfum" in s or "extrait" in s or "elixir" in s:
        return "Parfum / Extrait"
    if "cologne" in s or s == "edc":
        return "Cologne"
    if "perfume" in s or "fragrance" in s:
        return "Generic fragrance"
    return "Other"                              # body sprays, oils, deodorants, gift sets


df["product_type"] = df["Catatory"].apply(group_category)
print("Raw labels:", df["Catatory"].nunique(), "-> groups:", df["product_type"].nunique())
print(df["product_type"].value_counts())
log_step("Collapse category labels into 6 product types", df["Catatory"].notna().sum(), "Case, spelling and wording variants")

# 3.6 Brand: tidy whitespace; a missing brand is treated as 'Unbranded'
df["brand"] = df["Item_Name"].fillna("Unbranded").str.strip()
log_step("Missing brand set to 'Unbranded'", df["Item_Name"].isna().sum(), "Single row")

# 3.7 Location: keep the country (last part of 'City, State, Country')
df["country"] = df["itemLocation"].str.split(",").str[-1].str.strip()
df["ships_from_us"] = (df["country"] == "United States").astype(int)

# 3.8 Last update: parse dates (two month formats) and turn them into 'days since update'
dates = pd.to_datetime(df["lastUpdated"].str.replace(r" P[DS]T", "", regex=True), format="mixed", errors="coerce")
df["days_since_update"] = (dates.max() - dates).dt.total_seconds() / 86400
print("Unparsed or missing dates:", dates.isna().sum(), "| latest update:", dates.max())

# 3.9 Bottle size in ounces, parsed from the description (used for profiling, not clustering)
size = df["details"].str.extract(r"(\d+(?:\.\d+)?)\s*(?:fl\.?\s*)?oz", flags=re.IGNORECASE)[0].astype(float)
df["size_oz"] = size.where(size.between(0.3, 8.5))   # ignore implausible parses
print("Bottle size found for", df["size_oz"].notna().sum(), "listings")
```

Code A14. Notebook code cell 14

Source: COM7022 - Assignment/output/screenshots/appendix/cell14.png

COM7022_Clustering_25199053.ipynb | code cell 15 (notebook cell 21)

```
# 3.10 Transform the skewed measures. log(1 + x) keeps zero sales valid (log of 0 is undefined).
df["log_price"] = np.log1p(df["price"])
df["log_sold"] = np.log1p(df["sold"])

skew_table = pd.DataFrame({
    "skew_before": [skew(df["price"]), skew(df["sold"])],
    "skew_after_log": [skew(df["log_price"]), skew(df["log_sold"])],
}, index=["price", "sold"]).round(2)
print(skew_table)

# Outliers: count them with the 1.5 x IQR rule on the raw scale, but keep them.
# Best-selling listings are exactly what the business wants to find, so removing them would be wrong.
q1, q3 = df["sold"].quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)
print("Listings above the sold upper fence (" + str(round(upper_fence)) + "):", (df["sold"] > upper_fence).sum())
log_step("Log-transform price and sold", len(df), "Outliers kept; log scale limits their pull on distances")

cleaning_table = pd.DataFrame(cleaning_log)
save_table(cleaning_table, "table03_cleaning_log.csv")
print("Rows after cleaning:", len(df))
cleaning_table
```

Code A15. Notebook code cell 15

Source: COM7022 - Assignment/output/screenshots/appendix/cell15.png

COM7022_Clustering_25199053.ipynb | code cell 16 (notebook cell 22)

```
# Figure 3: distributions after the log transform
fig, axes = plt.subplots(1, 3, figsize=(13, 3.8))
axes[0].hist(df["log_price"], bins=35, color=MIDNIGHT_SOFT, edgecolor="white")
axes[0].set_title("log(1 + price)")
axes[1].hist(df["log_sold"], bins=35, color=TIFFANY_DEEP, edgecolor="white")
axes[1].set_title("log(1 + units sold)")
axes[2].bar(df["stock_level"].value_counts().sort_index().index,
            df["stock_level"].value_counts().sort_index().values, color=TEAL, edgecolor="white")
axes[2].set_title("Stock level (11 = more than 10)")
for ax in axes:
    ax.set_ylabel("Number of listings")
fig.tight_layout()
save_figure(fig, "fig03_transformed_distributions.png")
plt.show()
```

Code A16. Notebook code cell 16

Source: COM7022 - Assignment/output/screenshots/appendix/cell16.png

COM7022_Clustering_25199053.ipynb | code cell 17 (notebook cell 23)

```
# Descriptive statistics after cleaning (these are the numbers the models see, before scaling)
clean_desc = df[["price", "sold", "stock_level", "log_price", "log_sold", "days_since_update", "size_oz"]].describe().T
clean_desc["skewness"] = [skew(df[c].dropna()) for c in clean_desc.index]
clean_desc = clean_desc.round(2)
save_table(clean_desc, "table04_descriptive_statistics_clean.csv")
clean_desc
```

Code A17. Notebook code cell 17

Source: COM7022 - Assignment/output/screenshots/appendix/cell17.png

A · 4. Feature selection

COM7022_Clustering_25199053.ipynb | code cell 18 (notebook cell 25)

```
candidates = ["log_price", "log_sold", "stock_level", "days_since_update", "size_oz"]
corr = df[candidates].corr(method="spearman")

# Figure 4: Spearman correlation between candidate features
fig, ax = plt.subplots(figsize=(6.5, 5.2))
sns.heatmap(corr, annot=True, fmt=".2f", cmap=ARDEN_DIVERGING, vmin=-1, vmax=1,
            linewidths=1, linecolor="white", cbar_kws={"label": "Spearman correlation"}, ax=ax)
ax.set_title("Correlation between candidate features")
plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
save_figure(fig, "fig04_feature_correlation.png")
plt.show()
```

Code A18. Notebook code cell 18

Source: COM7022 - Assignment/output/screenshots/appendix/cell18.png

COM7022_Clustering_25199053.ipynb | code cell 19 (notebook cell 26)

```
# Compare feature sets on the same footing: K-Means silhouette for k = 2..6.
# Rows with missing values in optional features are filled with the median only for this comparison.
feature_sets = {
    "A: price + sold": ["log_price", "log_sold"],
    "B: price + sold + stock": ["log_price", "log_sold", "stock_level"],
    "C: B + days since update": ["log_price", "log_sold", "stock_level", "days_since_update"],
    "D: B + bottle size": ["log_price", "log_sold", "stock_level", "size_oz"],
    "E: B + days + size": ["log_price", "log_sold", "stock_level", "days_since_update", "size_oz"],
}

rows = []
for name, cols in feature_sets.items():
    data = df[cols].fillna(df[cols].median())
    X_try = StandardScaler().fit_transform(data)
    row = {"feature_set": name, "n_features": len(cols)}
    for k in range(2, 7):
        labels_try = KMeans(n_clusters=k, n_init=20, random_state=SEED).fit_predict(X_try)
        row["k=" + str(k)] = round(silhouette_score(X_try, labels_try), 3)
    rows.append(row)

feature_scan = pd.DataFrame(rows).set_index("feature_set")
save_table(feature_scan, "table05_feature_set_scan.csv")
feature_scan
```

Code A19. Notebook code cell 19

Source: COM7022 - Assignment/output/screenshots/appendix/cell19.png

COM7022_Clustering_25199053.ipynb | code cell 20 (notebook

```
FEATURES = ["log_price", "log_sold", "stock_level"]
scaler = StandardScaler()
X = scaler.fit_transform(df[FEATURES])
print("Feature matrix:", X.shape)
print("Means after scaling:", X.mean(axis=0).round(3))
print("Std after scaling:", X.std(axis=0).round(3))
```

Code A20. Notebook code cell 20

Source: COM7022 - Assignment/output/screenshots/appendix/cell20.png

A · 5. Principal Component Analysis

COM7022_Clustering_25199053.ipynb | code cell 21 (notebook cell 30)

```
pca = PCA(random_state=SEED)
X_pca_full = pca.fit_transform(X)
explained = pca.explained_variance_ratio_
loadings = pd.DataFrame(pca.components_.T, index=FEATURES,
                        columns=["PC" + str(i + 1) for i in range(len(FEATURES))]).round(3)
print("Explained variance ratio:", explained.round(3))
print("Cumulative:", explained.cumsum().round(3))
save_table(loadings, "table06_pca_loadings.csv")
loadings
```

Code A21. Notebook code cell 21

Source: COM7022 - Assignment/output/screenshots/appendix/cell21.png

COM7022_Clustering_25199053.ipynb | code cell 22 (notebook cell 31)

```
# Figure 5: explained variance (left) and feature loadings on PC1/PC2 (right)
X_2d = X_pca_full[:, :2]
fig, axes = plt.subplots(1, 2, figsize=(12, 4.3))
pcs = ["PC1", "PC2", "PC3"]
axes[0].bar(pcs, explained, color=[MIDNIGHT, MIDNIGHT_SOFT, GRID])
axes[0].plot(pcs, explained.cumsum(), color=TIFFANY_DEEP, marker="o", linewidth=2, label="Cumulative")
for i, v in enumerate(explained.cumsum()):
    axes[0].text(i, v + 0.03, str(round(v * 100)) + "%", ha="center", color=MIDNIGHT, fontsize=9)
axes[0].set_ylim(0, 1.12)
axes[0].set_ylabel("Share of variance explained")
axes[0].set_title("Explained variance")
axes[0].legend(loc="center right")

axes[1].scatter(X_2d[:, 0], X_2d[:, 1], s=8, color=NOISE_GREY, alpha=0.5)
for i, feature in enumerate(FEATURES):
    axes[1].arrow(0, 0, loadings.iloc[i, 0] * 2.5, loadings.iloc[i, 1] * 2.5, color=TIFFANY_DEEP,
                  width=0.02, head_width=0.12)
    axes[1].text(loadings.iloc[i, 0] * 2.9, loadings.iloc[i, 1] * 2.9, feature, color=MIDNIGHT,
                 fontsize=9, fontweight="bold", ha="center")
axes[1].set_xlabel("PC1 (" + str(round(explained[0] * 100)) + "%)")
axes[1].set_ylabel("PC2 (" + str(round(explained[1] * 100)) + "%)")
axes[1].set_title("Listings and feature directions (biplot)")
fig.tight_layout()
save_figure(fig, "fig05_pca_variance_biplot.png")
plt.show()
```

Code A22. Notebook code cell 22

Source: COM7022 - Assignment/output/screenshots/appendix/cell22.png

A · 6. K-Means

COM7022_Clustering_25199053.ipynb | code cell 23 (notebook cell 33)

```
k_values = range(2, 11)
kmeans_scan = []
for k in k_values:
    model = KMeans(n_clusters=k, n_init=20, random_state=SEED)
    labels_k = model.fit_predict(X)
    kmeans_scan.append({
        "k": k,
        "inertia": round(model.inertia_, 1),
        "silhouette": round(silhouette_score(X, labels_k), 3),
        "davies_bouldin": round(davies_bouldin_score(X, labels_k), 3),
        "calinski_harabasz": round(calinski_harabasz_score(X, labels_k), 1),
        "smallest_cluster": int(np.bincount(labels_k).min()),
    })
kmeans_scan = pd.DataFrame(kmeans_scan).set_index("k")
save_table(kmeans_scan, "table07_kmeans_k_scan.csv")
kmeans_scan
```

Code A23. Notebook code cell 23

Source: COM7022 - Assignment/output/screenshots/appendix/cell23.png

COM7022_Cluste

```
K_FINAL = 4
```

Code A24. Notebook code cell 24

Source: COM7022 - Assignment/output/screenshots/appendix/cell24.png

COM7022_Clustering_25199053.ipynb | code cell 25 (notebook cell 36)

```
# Figure 6: elbow (inertia) and silhouette against k
fig, ax1 = plt.subplots(figsize=(8.5, 4.3))
ax1.plot(kmeans_scan.index, kmeans_scan["inertia"], color=MIDNIGHT, marker="o", linewidth=2, label="Inertia")
ax1.set_xlabel("Number of clusters (k)")
ax1.set_ylabel("Inertia (within-cluster SS)", color=MIDNIGHT)
ax2 = ax1.twinx()
ax2.plot(kmeans_scan.index, kmeans_scan["silhouette"], color=TIFFANY_DEEP, marker="s", linewidth=2, label="Silhouette")
ax2.set_ylabel("Silhouette score", color=TIFFANY_DEEP)
ax2.grid(False)
ax2.spines["right"].set_visible(True)
ax1.axvline(K_FINAL, color=TIFFANY, linestyle="--", linewidth=1.5)
ax1.text(K_FINAL + 0.1, kmeans_scan["inertia"].max() * 0.95, "chosen k = " + str(K_FINAL), color=TIFFANY_DEEP,
         fontweight="bold")
ax1.set_title("Choosing k: elbow and silhouette")
fig.tight_layout()
save_figure(fig, "fig06_kmeans_elbow_silhouette.png")
plt.show()
```

Code A25. Notebook code cell 25

Source: COM7022 - Assignment/output/screenshots/appendix/cell25.png

COM7022_Clustering_25199053.ipynb | code cell 26 (notebook cell 37)

```
kmeans = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED)
df["kmeans"] = kmeans.fit_predict(X)

# Centroids back in original units, which is how a business reader thinks about them
centroids_scaled = pd.DataFrame(kmeans.cluster_centers_, columns=FEATURES)
centroids = pd.DataFrame(scaler.inverse_transform(kmeans.cluster_centers_), columns=FEATURES)
centroids["price_usd"] = np.expm1(centroids["log_price"]).round(2)
centroids["units_sold"] = np.expm1(centroids["log_sold"]).round(0)
centroids["stock_level"] = centroids["stock_level"].round(1)
centroids["listings"] = df["kmeans"].value_counts().sort_index().values
print(centroids[["price_usd", "units_sold", "stock_level", "listings"]])
```

Code A26. Notebook code cell 26

Source: COM7022 - Assignment/output/screenshots/appendix/cell26.png

COM7022_Clustering_25199053.ipynb | code cell 27 (notebook cell 38)

```
# Figure 7: K-Means clusters on the PCA plane, with a silhouette plot per cluster


def plot_clusters_pca(ax, labels, title, noise_label=None):
    unique = sorted(set(labels))
    for lab in unique:
        mask = labels == lab
        if noise_label is not None and lab == noise_label:
            ax.scatter(X_2d[mask, 0], X_2d[mask, 1], s=14, color=NOISE_GREY, marker="x", label="Noise")
        else:
            ax.scatter(X_2d[mask, 0], X_2d[mask, 1], s=12, alpha=0.75,
                       color=CLUSTER_COLOURS[lab % len(CLUSTER_COLOURS)], label="Cluster " + str(lab))
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_title(title)
    ax.legend(loc="best", fontsize=8, markerscale=1.5)


fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
plot_clusters_pca(axes[0], df["kmeans"].values, "K-Means (k = " + str(K_FINAL) + ") on the PCA plane")

sil_values = silhouette_samples(X, df["kmeans"])
y_lower = 10
for lab in range(K_FINAL):
    values = np.sort(sil_values[df["kmeans"].values == lab])
    y_upper = y_lower + len(values)
    axes[1].fill_betweenx(np.arange(y_lower, y_upper), 0, values, color=CLUSTER_COLOURS[lab], alpha=0.85)
    axes[1].text(-0.08, y_lower + len(values) / 2, str(lab), color=MIDNIGHT, fontweight="bold")
    y_lower = y_upper + 10
axes[1].axvline(sil_values.mean(), color=TIFFANY, linestyle="--", label="Mean = " + str(round(sil_values.mean(), 3)))
axes[1].set_xlabel("Silhouette value")
axes[1].set_ylabel("Listings, grouped by cluster")
axes[1].set_yticks([])
axes[1].set_title("Silhouette per listing")
axes[1].legend(loc="lower right")
fig.tight_layout()
save_figure(fig, "fig07_kmeans_clusters_silhouette.png")
plt.show()
```

Code A27. Notebook code cell 27

Source: COM7022 - Assignment/output/screenshots/appendix/cell27.png

A · 7. Hierarchical clustering

COM7022_Clustering_25199053.ipynb | code cell 28 (notebook cell 40)

```
linkage_methods = ["single", "complete", "average", "ward"]
distances = pdist(X)
linkage_results = {}
linkage_rows = []
for method in linkage_methods:
    Z = linkage(X, method=method)
    linkage_results[method] = Z
    coph_corr, _ = cophenet(Z, distances)
    labels_h = fcluster(Z, t=K_FINAL, criterion="maxclust")
    sizes = np.bincount(labels_h)[1:]
    linkage_rows.append({
        "linkage": method,
        "cophenetic_corr": round(coph_corr, 3),
        "silhouette_at_k": round(silhouette_score(X, labels_h), 3) if len(set(labels_h)) > 1 else np.nan,
        "cluster_sizes": "/".join(str(s) for s in sorted(sizes, reverse=True)),
    })
linkage_table = pd.DataFrame(linkage_rows).set_index("linkage")
save_table(linkage_table, "table08_linkage_comparison.csv")
linkage_table
```

Code A28. Notebook code cell 28

Source: COM7022 - Assignment/output/screenshots/appendix/cell28.png

COM7022_Clustering_25199053.ipynb | code cell 29 (notebook cell 41)

```
# Figure 8: dendrograms for the four linkages (truncated to the last 30 merges for readability)
fig, axes = plt.subplots(2, 2, figsize=(13, 8))
for ax, method in zip(axes.ravel(), linkage_methods):
    dendrogram(linkage_results[method], truncate_mode="lastp", p=30, ax=ax,
               color_threshold=0, above_threshold_color=MIDNIGHT_SOFT, no_labels=True)
    ax.set_title(method.capitalize() + " linkage (cophenetic r = "
                 + str(linkage_table.loc[method, "cophenetic_corr"]) + ")")
    ax.set_ylabel("Merge distance")
    ax.grid(axis="x", visible=False)
fig.tight_layout()
save_figure(fig, "fig08_dendrograms_four_linkages.png")
plt.show()
```

Code A29. Notebook code cell 29

Source: COM7022 - Assignment/output/screenshots/appendix/cell29.png

COM7022_Clustering_25199053.ipynb | code cell 30 (notebook cell 43)

```
# Ward: metrics across k, then the final cut at the same k as K-Means for a fair comparison
Z_ward = linkage_results["ward"]
ward_scan = []
for k in range(2, 11):
    labels_w = fcluster(Z_ward, t=k, criterion="maxclust")
    ward_scan.append({"k": k,
                      "silhouette": round(silhouette_score(X, labels_w), 3),
                      "davies_bouldin": round(davies_bouldin_score(X, labels_w), 3),
                      "calinski_harabasz": round(calinski_harabasz_score(X, labels_w), 1)})
ward_scan = pd.DataFrame(ward_scan).set_index("k")
save_table(ward_scan, "table09_ward_k_scan.csv")

# scikit-learn's AgglomerativeClustering gives the same Ward tree with 0-based labels
ward_model = AgglomerativeClustering(n_clusters=K_FINAL, linkage="ward")
df["ward"] = ward_model.fit_predict(X)
print("Ward cluster sizes:", np.bincount(df["ward"]))
print("Agreement with K-Means (Adjusted Rand Index):", round(adjusted_rand_score(df["kmeans"], df["ward"]), 3))
ward_scan
```

Code A30. Notebook code cell 30

Source: COM7022 - Assignment/output/screenshots/appendix/cell30.png

COM7022_Clustering_25199053.ipynb | code cell 31 (notebook cell 44)

```
# Figure 9: Ward dendrogram with the cut that gives K_FINAL clusters
cut_height = (Z_ward[-K_FINAL, 2] + Z_ward[-K_FINAL + 1, 2]) / 2
fig, ax = plt.subplots(figsize=(11, 4.5))
dendrogram(Z_ward, truncate_mode="lastp", p=40, ax=ax, color_threshold=cut_height,
           above_threshold_color=MIDNIGHT, no_labels=True)
ax.axhline(cut_height, color=TIFFANY_DEEP, linestyle="--", linewidth=2)
ax.text(ax.get_xlim()[1] * 0.99, cut_height * 1.04, "cut: " + str(K_FINAL) + " clusters",
        color=TIFFANY_DEEP, ha="right", fontweight="bold")
ax.set_title("Ward dendrogram and the chosen cut")
ax.set_ylabel("Merge distance (Ward)")
ax.grid(axis="x", visible=False)
save_figure(fig, "fig09_ward_dendrogram_cut.png")
plt.show()
```

Code A31. Notebook code cell 31

Source: COM7022 - Assignment/output/screenshots/appendix/cell31.png

A · 8. DBSCAN

COM7022_Clustering_25199053.ipynb | code cell 32 (notebook cell 46)

```
MIN_SAMPLES = 2 * len(FEATURES)
neighbours = NearestNeighbors(n_neighbors=MIN_SAMPLES).fit(X)
dist_k, _ = neighbours.kneighbors(X)
k_distance = np.sort(dist_k[:, -1])

# Figure 10: k-distance plot used to pick eps
fig, ax = plt.subplots(figsize=(8.5, 4.2))
ax.plot(k_distance, color=MIDNIGHT, linewidth=2)
for q, colour in [(0.90, TIFFANY), (0.95, TIFFANY_DEEP)]:
    ax.axhline(np.quantile(k_distance, q), color=colour, linestyle="--", linewidth=1.3,
               label=str(int(q * 100)) + "th percentile = " + str(round(np.quantile(k_distance, q), 2)))
ax.set_xlabel("Listings sorted by distance")
ax.set_ylabel("Distance to " + str(MIN_SAMPLES) + "th nearest neighbour")
ax.set_title("k-distance plot for choosing eps")
ax.legend(loc="upper left")
save_figure(fig, "fig10_dbscan_k_distance.png")
plt.show()
```

Code A32. Notebook code cell 32

Source: COM7022 - Assignment/output/screenshots/appendix/cell32.png

COM7022_Clustering_25199053.ipynb | code cell 33 (notebook cell 47)

```
# Grid search over eps and min_samples. Silhouette is computed on clustered points only,
# so it must be read together with the noise share and the number of clusters.
dbscan_rows = []
for eps in [0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.60, 0.70]:
    for min_s in [4, 6, 10, 15]:
        labels_d = DBSCAN(eps=eps, min_samples=min_s).fit_predict(X)
        n_clusters = len(set(labels_d)) - (1 if -1 in labels_d else 0)
        noise_share = (labels_d == -1).mean()
        mask = labels_d != -1
        sil = np.nan
        if n_clusters >= 2:
            sil = silhouette_score(X[mask], labels_d[mask])
        dbscan_rows.append({"eps": eps, "min_samples": min_s, "clusters": n_clusters,
                            "noise_pct": round(noise_share * 100, 1),
                            "silhouette_clustered": round(sil, 3) if not np.isnan(sil) else np.nan})
dbscan_grid = pd.DataFrame(dbscan_rows)
save_table(dbscan_grid.set_index(["eps", "min_samples"]), "table10_dbscan_grid.csv")
dbscan_grid.pivot(index="eps", columns="min_samples", values="clusters")
```

Code A33. Notebook code cell 33

Source: COM7022 - Assignment/output/screenshots/appendix/cell33.png

COM7022_Clustering_25199053.ipynb | code cell 34 (notebook cell 48)

```
dbscan_grid.pivot(index="eps", columns="min_samples", values="noise_pct")
```

Code A34. Notebook code cell 34

Source: COM7022 - Assignment/output/screenshots/appendix/cell34.png

COM7022_Clustering_25199053.ipynb | code cell 35 (notebook cell 49)

```
EPS_FINAL = 0.45
dbscan = DBSCAN(eps=EPS_FINAL, min_samples=MIN_SAMPLES)
df["dbscan"] = dbscan.fit_predict(X)
print("DBSCAN eps =", EPS_FINAL, "min_samples =", MIN_SAMPLES)
print(df["dbscan"].value_counts().sort_index())

# What do the noise points look like?  (They are the listings DBSCAN considers unusual.)
noise_profile = df.groupby(df["dbscan"] == -1)[["price", "sold", "stock_level"]].median()
noise_profile.index = ["clustered", "noise"]
noise_profile
```

Code A35. Notebook code cell 35

Source: COM7022 - Assignment/output/screenshots/appendix/cell35.png

COM7022_Clustering_25199053.ipynb | code cell 36 (notebook cell 50)

```
# Figure 11: DBSCAN result on the PCA plane
fig, ax = plt.subplots(figsize=(7.5, 5))
plot_clusters_pca(ax, df["dbscan"].values, "DBSCAN (eps = " + str(EPS_FINAL) + ", min_samples = "
                  + str(MIN_SAMPLES) + ")", noise_label=-1)
save_figure(fig, "fig11_dbscan_clusters.png")
plt.show()
```

Code A36. Notebook code cell 36

Source: COM7022 - Assignment/output/screenshots/appendix/cell36.png

A · 9. Model comparison and validation

COM7022_Clustering_25199053.ipynb | code cell 37 (notebook cell 53)

```python
def internal_scores(labels, ignore_noise=False):
    labels = np.asarray(labels)
    mask = np.ones(len(labels), dtype=bool)
    if ignore_noise:
        mask = labels != -1
    return {
        "silhouette": round(silhouette_score(X[mask], labels[mask]), 3),
        "davies_bouldin": round(davies_bouldin_score(X[mask], labels[mask]), 3),
        "calinski_harabasz": round(calinski_harabasz_score(X[mask], labels[mask]), 1),
    }


def bootstrap_stability(fit_function, n_runs=30, frac=0.8):
    # Fit on the full data once, then on random subsamples, and compare labels on the shared rows
    full_labels = fit_function(X)
    rng = np.random.default_rng(SEED)
    scores = []
    for run in range(n_runs):
        idx = rng.choice(len(X), size=int(frac * len(X)), replace=False)
        sub_labels = fit_function(X[idx])
        scores.append(adjusted_rand_score(full_labels[idx], sub_labels))
    return np.mean(scores), np.std(scores)


def fit_kmeans(data):
    return KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(data)


def fit_ward(data):
    return AgglomerativeClustering(n_clusters=K_FINAL, linkage="ward").fit_predict(data)


def fit_dbscan(data):
    return DBSCAN(eps=EPS_FINAL, min_samples=MIN_SAMPLES).fit_predict(data)


comparison_rows = []
for name, column, fit_function, ignore_noise in [
        ("K-Means", "kmeans", fit_kmeans, False),
        ("Hierarchical (Ward)", "ward", fit_ward, False),
        ("DBSCAN", "dbscan", fit_dbscan, True)]:
    labels = df[column].values
    scores = internal_scores(labels, ignore_noise=ignore_noise)
    stab_mean, stab_std = bootstrap_stability(fit_function)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    sizes = pd.Series(labels[labels != -1]).value_counts()
    row = {"model": name, "clusters": n_clusters, "noise_pct": round((labels == -1).mean() * 100, 1)}
    row.update(scores)
    row["stability_ARI_mean"] = round(stab_mean, 3)
    row["stability_ARI_sd"] = round(stab_std, 3)
    row["smallest_cluster_pct"] = round(sizes.min() / len(labels) * 100, 1)
    row["largest_cluster_pct"] = round(sizes.max() / len(labels) * 100, 1)
    comparison_rows.append(row)

comparison = pd.DataFrame(comparison_rows).set_index("model")
save_table(comparison, "table11_model_comparison.csv")
comparison
```

Code A37. Notebook code cell 37

Source: COM7022 - Assignment/output/screenshots/appendix/cell37.png

COM7022_Clustering_25199053.ipynb | code cell 38 (notebook cell 54)

```python
# Seed stability for K-Means: does the answer change with the random start?
seed_labels = [KMeans(n_clusters=K_FINAL, n_init=20, random_state=s).fit_predict(X) for s in range(10)]
seed_ari = [adjusted_rand_score(seed_labels[0], lab) for lab in seed_labels[1:]]
print("K-Means ARI across 10 seeds: min", round(min(seed_ari), 3), "mean", round(np.mean(seed_ari), 3))

# Sensitivity 1: min-max scaling instead of z-scores (as used in the lessons)
X_minmax = MinMaxScaler().fit_transform(df[FEATURES])
km_minmax = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(X_minmax)
print("ARI z-score vs min-max K-Means:", round(adjusted_rand_score(df["kmeans"], km_minmax), 3))

# Sensitivity 2: drop the top 1% sellers and re-fit; compare labels on the remaining rows
keep = df["sold"] < df["sold"].quantile(0.99)
X_trim = StandardScaler().fit_transform(df.loc[keep, FEATURES])
km_trim = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(X_trim)
print("ARI full vs without top 1% sellers:", round(adjusted_rand_score(df.loc[keep, "kmeans"], km_trim), 3))

sensitivity = pd.DataFrame({
    "check": ["K-Means across 10 seeds (mean ARI)", "z-score vs min-max scaling (ARI)", "with vs without top 1% sellers (ARI)"],
    "ARI": [round(np.mean(seed_ari), 3),
            round(adjusted_rand_score(df["kmeans"], km_minmax), 3),
            round(adjusted_rand_score(df.loc[keep, "kmeans"], km_trim), 3)],
}).set_index("check")
save_table(sensitivity, "table12_sensitivity_checks.csv")
sensitivity
```

Code A38. Notebook code cell 38

Source: COM7022 - Assignment/output/screenshots/appendix/cell38.png

COM7022_Clustering_25199053.ipynb | code cell 39 (notebook cell 55)

```
# Figure 12: side-by-side comparison of the three models
metrics_to_plot = [("silhouette", "Silhouette (higher = better)"),
                   ("davies_bouldin", "Davies-Bouldin (lower = better)"),
                   ("stability_ARI_mean", "Stability ARI (higher = better)")]
fig, axes = plt.subplots(1, 3, figsize=(13, 3.9))
model_colours = [MIDNIGHT, MIDNIGHT_SOFT, TEAL]
for ax, (metric, label) in zip(axes, metrics_to_plot):
    values = comparison[metric]
    bars = ax.bar(comparison.index, values, color=model_colours)
    for bar, v in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() * 1.02, str(v), ha="center",
                color=MIDNIGHT, fontsize=9, fontweight="bold")
    ax.set_title(label, fontsize=11)
    ax.set_ylim(0, max(values) * 1.25)
    plt.setp(ax.get_xticklabels(), rotation=20, ha="right")
    ax.grid(axis="x", visible=False)
fig.text(0.5, -0.06, "DBSCAN scores are computed on clustered points only ("
         + str(comparison.loc["DBSCAN", "noise_pct"]) + "% of listings labelled as noise).",
         ha="center", color=MUTE, fontsize=9)
fig.tight_layout()
save_figure(fig, "fig12_model_comparison.png")
plt.show()
```

Code A39. Notebook code cell 39

Source: COM7022 - Assignment/output/screenshots/appendix/cell39.png

A · 10. Final model and cluster profiles

COM7022_Clustering_25199053.ipynb | code cell 40 (notebook cell 58)

```
FINAL = "kmeans"
groups = df.groupby(FINAL)
profile = pd.DataFrame({
    "listings": groups.size(),
    "share_of_listings_pct": (groups.size() / len(df) * 100).round(1),
    "median_price_usd": groups["price"].median().round(2),
    "median_units_sold": groups["sold"].median().round(0),
    "mean_units_sold": groups["sold"].mean().round(0),
    "share_of_units_sold_pct": (groups["sold"].sum() / df["sold"].sum() * 100).round(1),
    "median_stock_level": groups["stock_level"].median(),
    "pct_last_one": groups["stock_level"].apply(lambda s: (s == 1).mean() * 100).round(1),
    "pct_ships_from_us": (groups["ships_from_us"].mean() * 100).round(1),
    "median_days_since_update": groups["days_since_update"].median().round(1),
})
top_type = groups["product_type"].agg(lambda s: s.value_counts().index[0])
top_type_share = groups["product_type"].agg(lambda s: round(s.value_counts(normalize=True).iloc[0] * 100, 1))
profile["top_product_type"] = top_type + " (" + top_type_share.astype(str) + "%)"
profile["top_brands"] = groups["brand"].agg(lambda s: ", ".join(s.value_counts().index[:3]))
save_table(profile, "table13_cluster_profiles.csv")
profile.T
```

Code A40. Notebook code cell 40

Source: COM7022 - Assignment/output/screenshots/appendix/cell40.png

COM7022_Clustering_25199053.ipynb | code cell 41 (notebook cell 59)

```
# Product-type mix per cluster (row percentages)
type_mix = pd.crosstab(df[FINAL], df["product_type"], normalize="index").mul(100).round(1)
save_table(type_mix, "table14_product_type_mix.csv")
type_mix
```

Code A41. Notebook code cell 41

Source: COM7022 - Assignment/output/screenshots/appendix/cell41.png

COM7022_Clustering_25199053.ipynb | code cell 42 (notebook cell 60)

```
# Figure 13: cluster profiles as standardised feature means (how each cluster differs from the average listing)
z_profile = pd.DataFrame(X, columns=["Price (log)", "Units sold (log)", "Stock level"])
z_profile[FINAL] = df[FINAL].values
z_means = z_profile.groupby(FINAL).mean()
z_means.index = ["Cluster " + str(i) for i in z_means.index]

fig, axes = plt.subplots(1, 2, figsize=(13, 4.3), gridspec_kw={"width_ratios": [1.1, 1]})
sns.heatmap(z_means, annot=True, fmt=".2f", cmap=ARDEN_DIVERGING, center=0, vmin=-2, vmax=2,
            linewidths=1, linecolor="white", cbar_kws={"label": "Mean z-score"}, ax=axes[0])
axes[0].set_title("Cluster centres relative to the average listing")
axes[0].set_ylabel("")

share = profile[["share_of_listings_pct", "share_of_units_sold_pct"]].copy()
share.index = ["Cluster " + str(i) for i in share.index]
x_pos = np.arange(len(share))
axes[1].bar(x_pos - 0.2, share["share_of_listings_pct"], width=0.4, color=MIDNIGHT_SOFT, label="Share of listings")
axes[1].bar(x_pos + 0.2, share["share_of_units_sold_pct"], width=0.4, color=TIFFANY_DEEP, label="Share of units sold")
axes[1].set_xticks(x_pos)
axes[1].set_xticklabels(share.index)
axes[1].set_ylabel("Percent")
axes[1].set_title("Listings vs units sold by cluster")
axes[1].legend(loc="upper left")
axes[1].grid(axis="x", visible=False)
fig.tight_layout()
save_figure(fig, "fig13_cluster_profiles.png")
plt.show()
```

Code A42. Notebook code cell 42

Source: COM7022 - Assignment/output/screenshots/appendix/cell42.png

COM7022_Clustering_25199053.ipynb | code cell 43 (notebook cell 61)

```
# Figure 14: distributions of price and units sold inside each cluster
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
order = sorted(df[FINAL].unique())
palette = [CLUSTER_COLOURS[i] for i in order]
sns.boxplot(data=df, x=FINAL, y="price", order=order, palette=palette, ax=axes[0], fliersize=2)
axes[0].set_title("Price by cluster")
axes[0].set_ylabel("Price (USD)")
sns.boxplot(data=df, x=FINAL, y="sold", order=order, palette=palette, ax=axes[1], fliersize=2)
axes[1].set_yscale("log")
axes[1].set_title("Units sold by cluster (log scale)")
axes[1].set_ylabel("Units sold")
sns.boxplot(data=df, x=FINAL, y="stock_level", order=order, palette=palette, ax=axes[2], fliersize=2)
axes[2].set_title("Stock level by cluster")
axes[2].set_ylabel("Stock level (11 = more than 10)")
for ax in axes:
    ax.set_xlabel("Cluster")
fig.tight_layout()
save_figure(fig, "fig14_cluster_distributions.png")
plt.show()
```

Code A43. Notebook code cell 43
Source: COM7022 - Assignment/output/screenshots/appendix/cell43.png

COM7022_Clustering_25199053.ipynb | code cell 44 (notebook cell 63)

```
# Save the frozen results ledger: cluster labels for every listing plus the headline numbers
df[["Item_Name", "details", "product_type", "price", "sold", "stock_level", "stock_imputed",
    "country", "kmeans", "ward", "dbscan"]].to_csv(OUTPUT_DIR / "cluster_assignments.csv", index=False)

ledger = {
    "rows_raw": int(len(raw)),
    "rows_clean": int(len(df)),
    "features": FEATURES,
    "k_final": K_FINAL,
    "dbscan_eps": EPS_FINAL,
    "dbscan_min_samples": MIN_SAMPLES,
    "pca_explained_pc1_pc2": [round(float(v), 3) for v in explained[:2]],
    "comparison": comparison.reset_index().to_dict(orient="records"),
    "sensitivity": sensitivity.reset_index().to_dict(orient="records"),
    "kmeans_ward_ari": round(adjusted_rand_score(df["kmeans"], df["ward"]), 3),
    "profiles": profile.reset_index().to_dict(orient="records"),
}
with open(OUTPUT_DIR / "results_ledger.json", "w") as f:
    json.dump(ledger, f, indent=2, default=str)
print("Results ledger saved to", OUTPUT_DIR / "results_ledger.json")
```

Code A44. Notebook code cell 44
Source: COM7022 - Assignment/output/screenshots/appendix/cell44.png

AI use declaration. AI tools were used to support structuring and proofreading. All analysis was executed, verified and interpreted by the author, and every reported figure was checked against the notebook output. *(Align wording with the Arden AI policy before submission.)*